

4. .md files (in order)

README.md

```
# TrangorgeOS

# TrangorgeOS

> A modern, high-performance bare-metal operating system built ground-up on a custom **Separated Tri-Partition Architecture**

---

## Overview & Philosophy

TrangorgeOS is built with a strict **from-scratch philosophy** (~35,000+ lines of code, actively expanding)

The project operates under an intensive development cycle aimed at producing a stable, bare-metal kernel

---

## Visuals & Screenshots

| Kernel Loading & Memory Allocation Testing | Kernel Base Resolution | Kernel in 1080p |
|:---:|:---:|:---:|
| ![Kernel Loading](kernelloading.png) | ![Kernel Base Res](kernel_loader_base_res.png) | ![Kernel 1080p](kernel_1080p.png) |

---

## Architectural Model: Separated Tri-Partition Architecture

TrangorgeOS decouples driver functionality and system privilege into 4 distinct operational layers (3 main components):

...

+-----+
|                USERSPACE                |
| - Applications & High-Level Libraries    |
+-----+
|                USER DRIVER SPACE (UDS)   |
| - High-risk / Peripherals (Fault-isolated, strict boundary) |
+-----+
|                DRIVER SPACE (DS)         |
| * Dynamic Drivers   : GPU & complex hardware |
| * Static Drivers    : Init & single-action hardware setup |
| * Library Drivers   : Inter-driver interface providers |
+-----+
|                KERNEL CORE / DRIVERS     |
| - Trusted Core Drivers (Network, USB stack base, Core MM) |
| - Fine-grained Kernel Policy Enforcement & Data Flow Control |
+-----+

...

1. **Kernel Core & Trusted Drivers:** Holds only maximum-trust drivers (e.g., base network, USB stack core)
2. **Driver Space (DS):** Modular driver execution environment with strict error margins:
   - **Dynamic Drivers:** Handle complex, stateful hardware (e.g., GPU control).
   - **Static Drivers:** Perform hardware initialization or non-exporting, single-purpose setup.
   - **Library Drivers:** Expose specialized interfaces (e.g., PCI, HDMI control) for other drivers to control.
3. **User Driver Space (UDS):** High-level peripheral drivers isolated at a safe distance from the core to prevent system crashes.
4. **Userspace & User Library Drivers:** Top-level space housing application runtime and user-level libraries.

---

## Current Subsystems & Refactoring Status

> **Notice:** The kernel is currently undergoing a major refactoring phase (modernization and safety audit)

- **Memory Management (MM):** Fully custom memory management subsystem and dynamic allocator written in C
- **Network & USB:** Network driver base operational; USB stack operating under a controlled bare-metal environment
- **Input & Display:** Built-in BIOS PS/2 fallback support; active development on modern USB input. Kernel
```

```

- **Multiprocessing & Interrupts:** SMP multi-core initialization active. APIC/IDT interrupt architecture
- **Binary Support:** Native execution support for `ELF` and custom `.bin` binaries with custom IPC inter

---

## Custom Versioning System

TrangorgeOS uses an in-house versioning scheme reflecting the precise state of development:

 $\text{Major} . \text{Changes} . \text{FixesPerChange} . \text{Iteration} \text{State}$ 

*Example: `v0.182.7.1a`*
- **`0` (Major):** Pre-release major build.
- **`182` (Changes):** Total cumulative feature changes introduced.
- **`7` (Fixes):** Bugfixes applied for the current change.
- **`1` (Iteration):** Single target architecture fully active.
- **`a` (State):** Alpha phase (`a` = initial work  $\rightarrow$  `b` = stabilization  $\rightarrow$  `g` =

### Scale Milestones:
- **Alpha:** Pre-release scale (up to ~60,000 LOC), active architectural building.
- **Beta:** System runs stably on physical hardware.
- **Gamma:** Final stabilization phase prior to Release `v1.0`.

---

## Git Branch Model & Workflow

The repository relies on a strict multi-tier branch hierarchy:

| Branch | Purpose |
|:---|:---|
| `init` | Conceptual structure, architecture blueprints, no active codebase. |
| `new` | Experimental features and isolated proof-of-concept tests. |
| `unstable` | Primary active development branch. |
| `stabilizing` | Refactoring, bug-hunting, and code modernizing before release. |
| `stable` | Tested, incremental updates with guaranteed stability. |
| `main` | Production showcase branch. |

---

## Project Roadmap

- [x] **Current Stage (`stabilizing` / Alpha v0.182.x):** Deep refactoring of MM allocator, scheduler mod
- [ ] **Milestone 1 (~1.5 Months):** Complete memory allocator refactoring, lock down current subsystem r
- [ ] **Milestone 2 (~4 Months):** Mature active drivers, write dedicated GPU/display drivers, initiate m
- [ ] **Milestone 3 (~1 Year):** Complete full bare-metal testing on physical hardware, establish stable

---

<p align="center">
  <i>TrangorgeOS – Building bare-metal systems from scratch, one commit at a time.</i>
</p>

```

TrangorgeOS — Architecture Documentation.markdown

```

# TrangorgeOS – Architecture Documentation

> **Document status:** Working draft 0.2
> **Scope:** System architecture, kernel layout, memory and allocator design, driver-space model, reposit
> **Sources:** The supplied project tree, the `unstable` branch, and the official project website [1] [2]

## 1. Project overview

TrangorgeOS is an operating system built from scratch around a **hybrid kernel**. The project currently t

The project is intended for environments where control, low resource usage, predictable deployment, and p

The repository is intentionally split into a kernel, driver-space components, standalone drivers, librari

## 2. Current state

According to the project author, the kernel is substantially implemented, the filesystem works but still

```


The intended rule is that a driver should not duplicate kernel mechanisms, while the kernel should not ne

6. Kernel subsystem map

Subsystem	Main location	Responsibility
Entry and diagnostics	`kernel/src/main.rs`, `serial.rs`, `vga_buffer.rs`, `testing.rs`	Boot-time en
ABI	`kernel/src/abi`	Allocation, memory, filesystem, and public API contracts.
CPU and interrupts	`cpu/`, `gdt.rs`, `interrupts.rs`	ACPI, LAPIC, GDT, trampolines, CPU state, and
Memory	`mm/`, `allocator/`	Physical and virtual memory, paging, heap, cache, protection, DMA, and a
Filesystem	`fs/`	MBR, disk access, ATA PIO, block drivers, FAT32, EXT4, TFS, and initialization.
Hardware drivers	`drivers/`, `pci.rs`	PCI, USB, HID, mass storage, host controllers, and glue code.
Graphics and terminal	`gfx/`, `terminal/`, `vga_buffer.rs`	Framebuffer, VGA, fonts, console, termin
Networking	`nic/`	Ethernet, packets, protocols, device abstractions, and VirtIO networking.
Installation and updates	`ctrlinstall/`	Repositories, indexes, manifests, dependency resolution, tr
Driver-space initialization	`driverspaceinit/`	Address-space setup, shared memory, initialization,

The memory subsystem bootstrap order is explicitly visible in `mm\_init()`: `arch\_memory\_init` → `paging_

7. Memory and allocator architecture

The memory subsystem is divided into physical memory, virtual memory, heap allocation, caching, DMA/conti

```text

Rust and C allocation APIs

|

v

Allocation API and FFI bridge

|

--> physical memory: bitmap, frames, PMM

--> virtual memory: VMM, mapping, pages, paging, TLB

--> heap: buddy allocator, optional slab allocator

--> special memory: contiguous allocation and DMA

--> cache: object cache and per-CPU support

--> protection: guard, isolation, permissions

--> process spaces: address\_space and mmap

...

### ### 7.1. Initialization

`mm\_init()` validates boot memory-map parameters and initializes the memory stack in a deliberate order.

### ### 7.2. Active heap path

The active heap path uses the buddy allocator. `heap.c` reserves a 256 MiB buddy region and maps physical

The buddy implementation tracks allocation orders, free lists, mapped pages, used/free statistics, and bu

### ### 7.3. Required allocator documentation

The next allocator specification should define ownership, alignment, blocking behavior, interrupt-context

## ## 8. Repository map

| Directory                | Role                                                                               |
|--------------------------|------------------------------------------------------------------------------------|
| `kernel/`                | Main kernel and its memory, filesystem, CPU, networking, ABI, and hardware layers. |
| `drivers/`               | Packages for concrete device drivers such as GPU, audio, camera, and Wacom.        |
| `driverspace/`           | Driver-space executable or runtime and its public API.                             |
| `driverspacelib/`        | Shared driver-space ABI, runtime, logging, memory, ring, audio, and block helpers. |
| `trangorgelibc/`         | Client-side library and syscall/ABI-facing facilities.                             |
| `comgrub/`               | GRUB boot component.                                                               |
| `comlimine/`             | Limine boot component.                                                             |
| `iso-builder/`           | ISO-image construction tool.                                                       |
| `triang-lang/`           | Lexer, parser, AST, semantic analysis, IR, and C/assembly code generation.         |
| `mp4_to_bmp/`            | Auxiliary media-conversion tool.                                                   |
| `targets/`               | Build-target configuration.                                                        |
| Docker files and scripts | Reproducible build and run environment.                                            |

## ## 9. Build and boot components

The repository contains both `comgrub` and `comlimine`, indicating two boot-component paths. The project

## ## 10. Current risks and open questions

The driver-space mechanism is present, but its long-term ABI and lifecycle policy are not yet fully specified.

The public source also requires a careful compile check around `kernel/src/driverspaceinit/init/init.rs`.

Architecture support should be tracked per target. Listing ARM64 and RISC-V on the website does not by itself

## ## 11. Summary

TrangorgeOS has a clear architectural direction: a hybrid kernel with a three-way driver split, a layered

The highest-value next steps are to stabilize the driver-space ABI, make the allocator contract explicit,

## ## References

[1]: <https://github.com/CTRL-F-org3/TrangorgeOS/tree/unstable> "TrangorgeOS – unstable branch"

[2]: <https://trangorgeos.website/> "Official TrangorgeOS website"

[3]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/driverspaceinit/init/init.rs> "TrangorgeOS – init.rs"

[4]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/driverspaceinit/init/service.rs> "TrangorgeOS – service.rs"

[5]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/alloc/heap/heap.c> "TrangorgeOS – heap.c"

[6]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/alloc/api/alloc.c> "TrangorgeOS – alloc.c"

[7]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/core/mm.c> "TrangorgeOS – mm.c"

# TrangorgeOS — TODO.md

## # TrangorgeOS – TODO

> **\*\*Purpose:\*\*** This file tracks the remaining work required to move TrangorgeOS from the current `unstable` branch to a stable release.  
>  
> **\*\*Status convention:\*\*** `TODO` means not started or not verified; `WIP` means partially implemented; `BL` means blocked.  
>  
> **\*\*Priority convention:\*\*** `P0` blocks reliable boot or basic correctness; `P1` is required for the next release; `P2` is a nice-to-have.

## ## 0. Current snapshot

| Area                            | Current state                                                                                        | Priority |
|---------------------------------|------------------------------------------------------------------------------------------------------|----------|
| Kernel boot and x86_64 baseline | Present, but requires repeatable build and boot verification                                         | P0       |
| Memory subsystem                | Layered implementation exists; initialization order is defined in `mm_init()`                        | P0       |
| Heap allocator                  | Buddy path is active; slab implementation exists but `HEAP_USE_SLAB` is `0`                          | P1       |
| Driver space                    | Separate address space, shared rings, initialization parameters, and services are present            | P0       |
| Driver-space ABI                | Implemented enough for experiments, not yet documented as stable                                     | P0       |
| Filesystem                      | FAT32/EXT4 and additional filesystem code are present; correctness and integration tests are pending | P1       |
| USB/PCI                         | Significant infrastructure exists; device coverage and hardware validation are incomplete            | P1       |
| Networking                      | NIC/VirtIO structures exist; network-driver work is explicitly a project priority                    | P1       |
| ARM64 and RISC-V                | Public target goals; feature parity and build/boot status must be established                        | P2       |
| Package manager                 | Kernel-side `ctrlinstall` modules exist; end-to-end package workflow needs completion                | P1       |
| Toolchain and ecosystem         | `triang-lang`, libraries, ISO tooling, and auxiliary tools exist at different levels                 | P1       |
| Release process                 | No published release; stable branch criteria are not yet formalized                                  | P1       |

## ## 1. P0 – immediate correctness blockers

### ### 1.1 Repair and compile-check driver-space initialization

**\*\*Status:\*\*** TODO  
**\*\*Area:\*\*** `kernel/src/driverspaceinit/init/init.rs`

The visible implementation creates a `Driverspace` structure with an address space, two ring physical address spaces, and a scratch page.

**\*\*Tasks:\*\***

- Add or remove scratch-page state consistently in the `Driverspace` structure and initialization path.
- Ensure the scratch page is allocated, zeroed, mapped, and released exactly once.
- Ensure `scratch\_view()` cannot return a pointer after the driver-space instance has been destroyed.
- Ensure every failure after partial allocation releases previously allocated frames and address-space mappings.
- Compile the kernel after the fix with the repository's documented target.

**\*\*Acceptance criteria:\*\*** The driver-space initialization module compiles without undefined fields or unre

### ### 1.2 Establish a reproducible baseline build

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** root Docker scripts, `kernel/`, `targets/`

Document and verify the one canonical build command. The repository contains Docker scripts, target confi

**\*\*Tasks:\*\***

- Verify the required Rust toolchain and external binaries.
- Run the Docker build from a clean environment.
- Record the produced kernel, image, and ISO paths.
- Run the image in an emulator and capture serial output.
- Add a short "Build and run" section to the root `README.md`.
- Make the build fail on warnings or errors that affect the boot artifact, where practical.

**\*\*Acceptance criteria:\*\*** A new contributor can clone the repository, run the documented commands, produce

### ### 1.3 Define the driver-space ABI versioning rules

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `kernel/src/driverspaceinit/abi/`, `driverspacelib/src/`

The protocol already contains `DsMsg`, ring structures, magic/version fields, and command identifiers. It

**\*\*Tasks:\*\***

- Document the binary layout, field widths, alignment, endianness, and reserved fields of `DsMsg` and rin
- Define the meaning of every command and every argument.
- Define error/status codes and whether negative values are stable ABI values.
- Define how unknown commands, malformed messages, full rings, and stale IDs are handled.
- Define the compatibility policy for `DS\_MAGIC`, `DS\_VERSION`, and command additions.
- Add compile-time size/alignment assertions on both kernel and driver-space sides.
- Add a protocol version document and a small ABI conformance test.

**\*\*Acceptance criteria:\*\*** The same ABI document can be used to implement a driver without reading kernel i

### ### 1.4 Make driver-space isolation explicit and testable

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `kernel/src/driverspaceinit/`, `kernel/src/mm/protection/`, `kernel/src/mm/process/`

The kernel creates a separate address space and maps shared pages into it. The security and failure-isola

**\*\*Tasks:\*\***

- Test that driver space cannot read arbitrary kernel virtual addresses.
- Test that driver-space mappings are limited to explicitly granted RAM and MMIO regions.
- Validate permissions for shared rings, initialization parameters, RAM grants, and MMIO grants.
- Reject overlapping, unaligned, out-of-range, and double-freed grants.
- Add a driver crash or invalid-command test and verify that kernel execution continues.
- Define whether and how a failed driver can be restarted or detached.

**\*\*Acceptance criteria:\*\*** Negative tests demonstrate that unauthorized mappings and malformed service requ

## ## 2. P1 – kernel and hardware reliability

### ### 2.1 Complete the allocator specification and test suite

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/mm/alloc/`

The active heap path uses buddy allocation. Slab code is present but disabled with `HEAP\_USE\_SLAB 0`. The

**\*\*Tasks:\*\***

- Document physical-frame, virtual-mapping, heap, buddy, slab, DMA, and contiguous-memory responsibilities
- Add tests for zero-size allocations, alignment, large allocations, `krealloc`, double free, invalid fre
- Test buddy split and coalescing across every supported order.
- Test page mapping rollback when a physical-frame allocation fails partway through.
- Verify `heap\_usable\_size()` for active and invalid pointers.

- Decide whether slab should remain disabled, be enabled for small objects, or be redesigned.
- Add allocator statistics and leak checks to a repeatable debug test.
- Define which allocator functions are safe during interrupt context and early boot.

**\*\*Acceptance criteria:\*\*** The allocator passes deterministic unit/stress tests and its public contract des

### ### 2.2 Finish filesystem correctness and integration tests

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/fs/`

The tree contains MBR, disk, ATA PIO, block-driver, FAT32, EXT4, TFS, and filesystem initialization code.

**\*\*Tasks:\*\***

- Define the supported FAT32 and EXT4 feature subset.
- Test mounting, directory creation, file creation, reads, writes, truncation, append, rename, delete, and
- Test unaligned reads/writes and operations spanning multiple blocks.
- Test corrupted superblocks, invalid MBR data, short reads, failed writes, and unexpected device removal
- Define and test filesystem locking and concurrent access rules.
- Separate experimental TFS behavior from the stable filesystem API.
- Add image-based regression fixtures for each supported filesystem.

**\*\*Acceptance criteria:\*\*** Filesystem tests pass on generated disk images and data remains correct after un

### ### 2.3 Stabilize PCI and USB discovery

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/drivers/`, `kernel/src/pci.rs`

USB core, EHCI/XHCI, HID, CDC, and mass-storage structures exist. The next step is reliable enumeration,

**\*\*Tasks:\*\***

- Document PCI configuration access and BAR discovery.
- Verify MMIO BAR validation and alignment.
- Complete USB device, configuration, interface, endpoint, and speed handling.
- Add enumeration tests with recorded or emulated descriptor sets.
- Test device removal, reset, stalled endpoints, transfer timeouts, and controller errors.
- Define which USB classes remain in kernel space and which move to driver space.
- Connect mass-storage discovery to the block-device registry and filesystem layer.

**\*\*Acceptance criteria:\*\*** A supported USB storage device and a supported HID device can be enumerated repe

### ### 2.4 Build a functional network-driver milestone

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/nic/`

The website identifies network-driver development as an important contribution area. The repository conta

**\*\*Tasks:\*\***

- Define the minimum supported NIC and VirtIO feature set.
- Complete RX/TX queue setup, descriptor ownership, interrupt or polling behavior, and recovery.
- Add packet-buffer ownership and lifetime rules.
- Implement and test a minimal Ethernet/ARP/IPv4 path, or explicitly document the current scope.
- Add checksum, MTU, malformed-packet, and queue-exhaustion tests.
- Add an emulator-based network test with captured packets.
- Define how network drivers are exposed to driver space.

**\*\*Acceptance criteria:\*\*** The selected NIC can transmit and receive validated packets in an emulator or te

### ### 2.5 Define and test audio driver integration

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/audio/`, `drivers/audiodriver/`, `driverspacelib/src/audio.rs`

Audio code spans kernel services, driver-space helpers, JACK-related bindings, and driver code. The owner

**\*\*Tasks:\*\***

- Define which audio operations are kernel services and which belong to the audio driver.

- Document buffer ownership, physical-page grants, sample format, rate, channel count, and duration units.
- Test `AudioInfo`, `AudioPlay`, `AudioStop`, and amplifier controls with invalid buffers and lengths.
- Verify that audio buffers cannot be used after revocation.
- Decide whether JACK integration is a development dependency or part of the target runtime.
- Add a minimal deterministic tone test in an emulator or supported host environment.

**\*\*Acceptance criteria:\*\*** Audio requests use a documented ABI, invalid grants are rejected, and the driver

### ### 2.6 Formalize bootloader and ISO output

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `comgrub/`, `comlimine/`, `iso-builder/`

The project contains GRUB and Limine-related boot components and a separate ISO builder. Their supported

**\*\*Tasks:\*\***

- Choose the primary supported boot path for the next milestone.
- Document the difference between GRUB and Limine images.
- Verify linker scripts, kernel load addresses, entry points, memory-map handoff, and initrd placement.
- Add ISO-content validation before boot.
- Add emulator smoke tests for each supported boot path.
- Record serial and graphical output expected at each milestone.

**\*\*Acceptance criteria:\*\*** The canonical ISO boots through the documented bootloader and fails with a usefu

## ## 3. P1 – code quality and integration

### ### 3.1 Define the kernel ABI boundary

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `kernel/src/abi/`, `trangorgellibc/src/abi/`

The kernel ABI and client-side library must agree on types, error values, syscall numbers, pointer rules,

**\*\*Tasks:\*\***

- Inventory every public ABI type and function.
- Define integer widths, structure layout, alignment, ownership, and lifetime rules.
- Define pointer validation and user/kernel buffer-copy rules.
- Synchronize error handling between kernel and `trangorgellibc`.
- Add generated or checked bindings where possible.
- Add ABI compatibility tests.

**\*\*Acceptance criteria:\*\*** A client library can be rebuilt against the documented ABI and pass layout and b

### ### 3.2 Define driver lifecycle and registration

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `driverspace/src/`, `driverspacelib/src/driver.rs`, `kernel/src/driverspaceinit/`

The project needs one lifecycle model for discovery, initialization, registration, attachment, running, o

**\*\*Tasks:\*\***

- Define driver states and allowed transitions.
- Define device identity, vendor/class fields, capabilities, and ownership.
- Complete driver registration and device attachment commands or remove unused commands from the public A
- Define resource acquisition and release for IRQs, MMIO, DMA, and shared memory.
- Define shutdown ordering and timeout handling.
- Add a mock driver used in CI to exercise the complete lifecycle.

**\*\*Acceptance criteria:\*\*** A mock driver can register, attach to a device, request resources, handle a requ

### ### 3.3 Reduce duplicated or experimental paths

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** whole repository

Several components appear to contain experimental or partially connected paths. These should be clearly l

**\*\*Tasks:\*\***

- Mark unused driver initialization helpers and experimental commands.
- Remove stale generated output from source-controlled directories where appropriate.
- Document intentionally empty or placeholder modules.
- Run formatting and static checks for Rust, C, Zig, and assembly sources where tools are available.
- Remove IDE metadata from production source directories unless intentionally retained.
- Record known warnings and decide whether each is acceptable or must be fixed.

**\*\*Acceptance criteria:\*\*** The repository has a documented list of experimental components, no unexplained

## ## 4. P2 – architecture portability

### ### 4.1 Separate architecture-independent and architecture-specific code

**\*\*Status:\*\*** TODO

**\*\*Area:\*\*** `kernel/src/mm/arch/x86\_64/`, CPU, paging, boot, targets

The current memory code visibly contains x86\_64-specific implementations. Portability requires clear inte

**\*\*Tasks:\*\***

- Define architecture-independent interfaces for page tables, TLB operations, interrupts, timers, CPU sta
- Move x86\_64-only constants and operations behind architecture modules.
- Define target-specific linker scripts and boot parameters.
- Add architecture capability tables for page size, address width, cache behavior, and DMA constraints.
- Add compile-only CI checks for ARM64 and RISC-V targets.

**\*\*Acceptance criteria:\*\*** Architecture-independent kernel code compiles without importing x86\_64 implement

### ### 4.2 ARM64 milestone

**\*\*Status:\*\*** BLOCKED by architecture abstraction work

- Define the ARM64 target specification.
- Implement or stub the boot entry and exception vector.
- Implement page-table and TLB operations.
- Implement interrupt-controller integration.
- Port the memory bootstrap and serial diagnostics.
- Boot a minimal ARM64 image in an emulator.

**\*\*Acceptance criteria:\*\*** A documented ARM64 smoke test reaches kernel initialization and reports memory-s

### ### 4.3 RISC-V milestone

**\*\*Status:\*\*** BLOCKED by architecture abstraction work

- Define the RISC-V target specification.
- Implement boot entry, trap handling, paging mode, and timer setup.
- Port serial diagnostics and memory initialization.
- Boot a minimal RISC-V image in an emulator.

**\*\*Acceptance criteria:\*\*** A documented RISC-V smoke test reaches kernel initialization and reports memory-

## ## 5. P2 – package manager and user ecosystem

### ### 5.1 Complete `ctrlinstall`

**\*\*Status:\*\*** WIP

**\*\*Area:\*\*** `kernel/src/ctrlinstall/`

The repository contains repository fetching, indexes, manifests, dependency resolution, transactions, dif

**\*\*Tasks:\*\***

- Define repository and package formats.
- Define signatures, trust roots, hashes, and rollback behavior.
- Complete dependency resolution and conflict diagnostics.
- Test interrupted transactions and power-loss recovery.
- Add install, remove, update, search, and list interfaces.
- Document how driver packages and kernel-compatible packages are separated.

**\*\*Acceptance criteria:\*\*** A package can be fetched, verified, installed, listed, upgraded, rolled back, and

### ### 5.2 Stabilize `trangorgelibs`

**\*\*Status:\*\*** TODO

- Inventory implemented versus placeholder modules.
- Define syscall wrappers and error semantics.
- Add memory, I/O, synchronization, and ABI tests.
- Document which interfaces are safe for applications and which are experimental.

### ### 5.3 Advance `triang-lang`

**\*\*Status:\*\*** WIP

- Document the language syntax and project format.
- Define the AST and IR stability policy.
- Add parser and semantic-analysis tests.
- Add deterministic code-generation tests for C and assembly.
- Define the x86\_64 output ABI.
- Decide which language features are required for system tools.

## ## 6. P3 – tools, documentation, and community

### ### 6.1 Documentation structure

**\*\*Status:\*\*** WIP

- Move the architecture documents into a stable `docs/` directory.
- Add a contributor guide and development-environment guide.
- Add a boot-flow diagram and driver-space sequence diagram.
- Add per-subsystem pages for memory, filesystem, USB, network, ABI, and package management.
- Keep English as the reference language or define an explicit bilingual-document policy.
- Add a changelog policy for ABI and boot-image changes.

### ### 6.2 Code editor and developer tools

**\*\*Status:\*\*** TODO

- Define the dedicated editor's scope and minimum viable feature set.
- Decide whether the editor is part of the OS image or a host-side development tool.
- Document project files, build integration, syntax support, and debugger/serial integration.
- Connect editor workflows to `triang-lang` and package-management tools.

### ### 6.3 Contribution and release process

**\*\*Status:\*\*** TODO

- Define branch policy for `main`, `unstable`, and the future stable branch.
- Define release criteria and a versioning scheme.
- Add CI for formatting, compilation, ABI tests, allocator tests, filesystem-image tests, and emulator boot tests.
- Add issue templates for kernel bugs, driver bugs, build failures, and documentation gaps.
- Document the AGPLv3 licensing requirements and contribution expectations.

## ## 7. Suggested milestone order

The recommended order is intentionally dependency-driven:

1. Repair and compile-check driver-space initialization.
2. Establish a reproducible x86\_64 build, ISO, and emulator boot test.
3. Freeze and test the driver-space ABI at an experimental version.
4. Add isolation and invalid-request tests.
5. Finish allocator, filesystem, USB, and network regre

... (truncated, file is very long)

## TrangorgeOS — dokumentacja architektury.markdown

# TrangorgeOS – dokumentacja architektury

> **\*\*Status dokumentu:\*\*** wersja robocza 0.1  
> **\*\*Zakres:\*\*** architektura systemu, podział jądra, pamięć i alokator, model sterowników oraz mapa repozytorium  
> **\*\*Źródła:\*\*** drzewo projektu przekazane przez autora, kod gąździ `unstable` oraz oficjalna strona projektu

### ## 1. Cel projektu

TrangorgeOS jest systemem operacyjnym budowanym od podstaw. Projekt ma docelowo obsługiwać kilka architek

Kod systemu jest wielojęzyczny, ale należy rozróżnić stan bieżący od planów. Publiczna gałąź `unstable`

Dokumentacja ma dwa cele. Po pierwsze, ma być \*\*mapą orientacyjną dla autora\*\*, który rozwija wiele pods

## ## 2. Stan projektu

Według opisu autora jądro jest już zaimplementowane, system plików działa, choć nadal wymaga poprawek, a

Poniższa tabela rozdziela informacje potwierdzone przez autora, twierdzenia oficjalnej strony oraz fakty

| Obszar           | Potwierdzone                                                                         | Wniosek ze struktury                                             |
|------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Architektura CPU | x86_64, później ARM64 i RISC-V                                                       | Obecne artefakty wskazują przede wszystkim na x86_64             |
| Typ jądra        | Hybrydowe                                                                            | Wydzielenie `driverspace` i `driverspacelib` wspiera taki model. |
| Języki           | Rust w kernelu; strona wymienia Ada/SPARK, C i język własny jako planowany ekosystem | GitHub                                                           |
| System plików    | Działa, ale ma problemy                                                              | W `kernel/src/fs` znajdują się obsługa dysku, MBR, FAT32, ex     |
| Alokator         | Około 8 tys. linii według autora                                                     | Kod potwierdza sterowanie oparte obecnie na buddy allocatorze    |
| Sterowniki       | Rozwijane, jeszcze nieukończone                                                      | Repozytorium zawiera sterowniki GPU, audio, kamery, Wac          |

## ## 3. Model wysokopoziomowy

System można obecnie opisać jako kilka współpracujących warstw:

```
```text
+-----+
| Programy, narzędzia i biblioteki użytkowe |
| triang-lang, trangorgelibc, mp4_to_bmp, narzędzia ISO |
+-----+
| Driver space | Biblioteki i kontrakty ABI |
| driverspace | driverspacelib, trangorgelibc |
| rejestracja i wykonywanie | syscall, typy, pamięć, I/O |
| sterowników | |
+-----+
| Kernel space | CPU, MM, FS, NIC, USB, PCI, |
| podstawowe mechanizmy | przerwania, grafika, terminal |
+-----+
| Sprzęt i firmware | x86_64; w przyszłości ARM64, |
| | RISC-V |
+-----+
```
```

Diagram jest zgodny z publicznym opisem projektu [2], natomiast szczegóły komunikacji są już częściowo

## ## 4. Trójkątna struktura sterowników

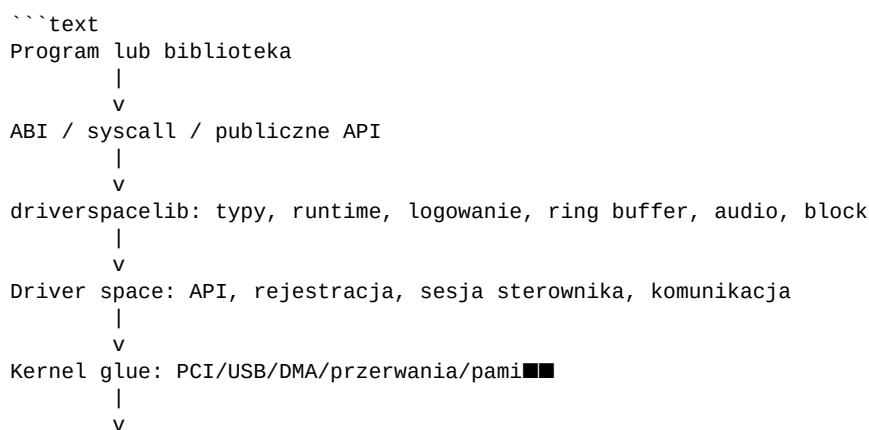
Oficjalna strona definiuje trójkątną jako \*\*kernel space\*\*, \*\*driver space\*\* i \*\*user space\*\* [2]. Kod

1. \*\*Kernel space\*\* – kod, który musi działać z najwyższymi uprawnieniami i bezpośrednim dostępem do me
2. \*\*Driver space\*\* – wydzielone środowisko dla sterowników, uruchamiane i obsługiwane przez mechanizmy
3. \*\*User space / warstwa kliencka\*\* – programy i biblioteki korzystające z usług sterowników przez stab

### ### 4.1. Przepływ danych sterownika

W aktualnym kodzie komunikacja kernel ↔ driver space odbywa się przez dwa pierścienie: `k2d` (kernel-to-

Przykładowy przepływ powinien wyglądać następująco:



Kontroler i urządzenie sprzętowe  
``

Najważniejsza zasada architektoniczna brzmi: \*\*sterownik nie powinien powielać logiki jądra, a jądro nie

### ### 4.2. Elementy modelu widoczne w repozytorium

| Element                          | Lokalizacja                  | Rola                                                               |
|----------------------------------|------------------------------|--------------------------------------------------------------------|
| API driver space                 | `driverspace/src/api.rs`     | Publiczne punkty wejścia i                                         |
| Biblioteka driver space          | `driverspacelib/src`         | ABI, runtime, logowanie, pamięć, ring buffer, audio,               |
| Inicjalizacja po stronie kernela | `kernel/src/driverspaceinit` | Wejście, komunikacja, komendy inicja                               |
| Pakiety sterowników              | `drivers/*`                  | Konkretne sterowniki GPU, audio, kamery i urządzenia wskazujących. |
| Interfejsy klienta               | `trangorgelibc/src`          | Typy ABI, błądy, tabela kernela, syscall oraz warstwy I/O,         |

## ## 5. Architektura jądra

Główny kod jądra znajduje się w `kernel/src`. Z punktu widzenia odpowiedzialności można go podzielić na

| Podsystem                   | Katalogi / pliki                                      | Odpowiedzialność                                                           |
|-----------------------------|-------------------------------------------------------|----------------------------------------------------------------------------|
| Punkt wejścia i diagnostyka | `main.rs`, `serial.rs`, `vga_buffer.rs`, `testing.rs` | Start systemu, wy                                                          |
| ABI                         | `abi/`                                                | Publiczne kontrakty alokacji, pamięci, systemu plików i API.               |
| CPU i przerwania            | `cpu/`, `gdt.rs`, `interrupts.rs`                     | ACPI, LAPIC, GDT, trampoliny, wejścia przerwa                              |
| Pamięć                      | `mm/`, `allocator/`                                   | Pamięć fizyczna i wirtualna, mapowanie, stronicowanie, ochrona, cache,     |
| Sterowniki sprzętowe        | `drivers/`, `pci.rs`                                  | PCI, USB, urządzenia HID, mass storage, host controllers                   |
| System plików               | `fs/`                                                 | MBR, dyski, ATA PIO, warstwa blokowa, FAT32, ext4, TFS i inicjalizacja FS. |
| Grafika i konsola           | `gfx/`, `terminal/`, `vga_buffer.rs`                  | Framebuffer, VGA, font, konsola, terminal or                               |
| Sieć                        | `nic/`                                                | Ethernet, pakiety, protokoły, urządzenia i VirtIO network queues.          |
| Instalacja i aktualizacje   | `ctrlinstall/`                                        | Repozytorium, indeksy, manifesty, rozwiązywanie zależno                    |
| Driver space                | `driverspaceinit/`                                    | Uruchamianie, komunikacja i usługi dla wydzielonej przestrzeni ster        |
| Audio                       | `audio/`                                              | Integracja audio po stronie kernela, w szczególności warstwa jack.         |

### ### 5.1. Zasady zależności

Zależności powinny płynąć od warstw wyższych do niższych przez jawne interfejsy, a nie przez przypadkow

Warto traktować katalogi `abi`, `driverspaceinit`, `driverspacelib` i `trangorgelibc` jako \*\*granice arch

## ## 6. Alokator i podsystem pamięci

Kod `kernel/src/mm` implementuje wielowarstwowy podsystem pamięci, a nie pojedynczą funkcję `alloc`. Roz

```text

Warstwy pamięci TrangorOS

API Rust / C i mostek FFI

```
|
v
mm/api.rs, mm/ffi.rs, mm/mm_bridge.rs
|
+--> pamięć fizyczna: pmm, bitmap, frame
|
+--> pamięć wirtualna: vmm, mapping, page, paging, tlb
|
+--> sterata: heap, buddy, slab
|
+--> pamięć specjalna: contiguous, dma
|
+--> cache i lokalność CPU: cache, object_cache, per_cpu
|
+--> przestrzeń procesów i ochrona: address_space, mmap,
guard, isolation, permissions
...
```

6.1. Pamięć fizyczna

Warstwa `mm/alloc/physical` zawiera bitmapę, ramki i PMM. Jej zadaniem jest śledzenie dostępnych jednost

6.2. Pamięć wirtualna i stronicowanie

Warstwy `virtual`, `paging`, `arch/x86\_64` oraz `process` wskazują na rozdzielenie mapowania adresów, tak

6.3. Sterta, buddy i slab

W bieżącej gałęzi `unstable` aktywnie ściskane steraty jest `**buddy allocator**`. `heap_init()` zawsze inicjuje

Kod `buddy.c` potwierdza przydział bloków według rzędu, listy wolnych bloków, mapowanie stron oraz koalescencję

Praktyczny podział odpowiedzialności powinien być następujący:

| Komponent | Zadanie |
|------------------------|--|
| PMM / bitmapa / frame | Przydział ramek fizycznych. |
| Buddy | Zarządzanie większymi, potęgami obszarów pamięci i ewentualna koalescencja bloków. |
| Heap | Wystawienie ogólnego interfejsu alokacji steraty. |
| Slab | Wydajny przydział obiektów o określonym rozmiarze. |
| DMA / contiguous | Przydział obszarów spełniających wymagania sprzętu. |
| Cache / per-CPU | Ograniczenie kosztu blokad i poprawa lokalności. |
| Protection / isolation | Kontrola praw dostępu oraz izolacja przestrzeni. |

6.4. Kontrakt, który warto dopisać do kodu

Dokumentacja alokatora powinna w przyszłości odpowiedzieć na sześć pytań: kto jest właścicielem przydzielonego

7. Sterowniki sprzętowe

Katalog `drivers` zawiera osobne pakiety dla różnych klas urządzeń. Aktualna mapa wygląda następująco.

| Pakiet | Przeznaczenie |
|----------------------------------|--|
| <code>amdgpu_driver</code> | Sterownik GPU AMD. |
| <code>intelgpu_driver</code> | Sterownik GPU Intel. |
| <code>netcam_driver</code> | Sterownik kamery sieciowej lub urządzenia tego typu. |
| <code>wacomgraphic_driver</code> | Sterownik urządzenia graficznego Wacom. |
| <code>audiodriver</code> | Sterownik audio; zawiera Rust, integrację jack oraz moduły Odin dla mikrofonu i głośników. |

W jądrze znajdują się ponadto ogólne elementy obsługi USB i PCI. USB jest podzielone na warstwy core, host

Warstwa sieciowa `kernel/src/nic` zawiera abstrakcje urządzenia i sterownika, Ethernet, pakiety, protokoły

8. Boot, budowanie i narzędzia

Projekt zawiera dwa widoczne warianty elementu startowego: `comgrub` oraz `comlimine`. Pierwszy zawiera `iso-builder` jest osobnym narzędziem do tworzenia obrazu ISO. Składa się z modułów Oberonowych (`.Mod`) `targets/x86_64-unknown-none` oraz `kernel/x86_64-kernel.json` wskazują na obecny profil bare-metal dla x86_64

`triang-lang` jest osobnym kompilatorem lub językiem projektowym. Zawiera lexer, parser, AST, analizę semantyczną

9. Mapa repozytorium

| Katalog | Znaczenie |
|------------------------------|---|
| <code>kernel/</code> | Główne jądro systemu, jego ABI, pamięć, FS, CPU, sieć i sterowniki. |
| <code>drivers/</code> | Osobne pakiety konkretnych sterowników urządzeń. |
| <code>driverspace/</code> | Program lub środowisko driver space oraz jego API. |
| <code>driverspacelib/</code> | Biblioteka wspólna dla sterowników działających w driver space. |
| <code>trangorgelibc/</code> | Biblioteka klienta i warstwa ABI/syscall. |
| <code>comgrub/</code> | Wariant komponentu startowego z GRUB. |
| <code>comlimine/</code> | Wariant komponentu startowego z Limine. |
| <code>iso-builder/</code> | Narzędzie tworzące obraz ISO. |
| <code>triang-lang/</code> | Język/kompilator z generatorami C i ASM. |
| <code>mp4_to_bmp/</code> | Narzędzie pomocnicze do konwersji MP4 do BMP. |
| <code>targets/</code> | Konfiguracje targetów kompilacji. |
| <code>Dockerfile</code> | i skrypty Docker Powtarzalne środowisko budowania i uruchamiania. |

10. Zasady utrzymania dokumentacji

Kiedy nowy podsystem powinien mieć krótki opis odpowiedzialności, publiczne wejście, zależności oraz interfejsy

Warto prowadzić dokumentację w układzie odpowiadającym repozytorium, ale nie kopiować bezpośrednio całości

Dla sterownika przydatny jest stały szablon:

```

Pole	Treść do uzupełnienia
Nazwa	Nazwa urządzenia i pakietu.
Przestrzeń wykonania	Kernel space, driver space albo user space.
Magistrala	PCI, USB, VirtIO lub inna.
Zalecenia	Pamięć, DMA, przerwania, kolejki, ABI.
Punkt inicjalizacji	Funkcja lub komenda uruchamiająca sterownik.
API	Operacje udostępniane klientom.
Ograniczenia	Brakujące funkcje, problemy i zagrożenia sprzętowe.
Status	Projektowany, eksperymentalny, działający lub stabilny.

```

11. Otwarte pytania techniczne

Poniższe kwestie powinny zostać doprecyzowane w następnej wersji dokumentacji, ponieważ nie da się ich

1. Jakie są docelowe granice uprawnień oraz które funkcje z protokołu driver space są już stabilne?
2. Kod potwierdza osobne przestrzenie adresowe, ale trzeba doprecyzować pełny cykl życia procesu/obrazu d
3. Jaki jest formalny format komunikatów między kernel-em a driver space?
4. Jak wygląda cykl życia sterownika: discovery, init, start, stop, reset, unload i obsługa awarii?
5. Czy buddy, slab i heap tworzą jeden mechanizm alokacji, czy są niezależnymi allocatorami dla różnych t
6. Czy `DsMsg` i komendy z `kernel/src/driverspaceinit/abi/abi.rs` mają już gwarantowaną kompatybilność
7. Jakie są minimalne wymagania dla implementacji ARM64 i RISC-V?
8. Które sterowniki są obecnie działające na prawdziwym sprzęcie, a które są dopiero szkieletem?

12. Podsumowanie

TrangorgeOS ma już wyraźny podział na jądro, przestrzeń sterowników, biblioteki, pakiety urządzeń, narzędzia

Najważniejszą wartością kolejnej iteracji dokumentacji będzie opisanie formalnych kontraktów: ABI, komunikacji

Referencje

[1]: <https://github.com/CTRL-F-org3/TrangorgeOS/tree/unstable> "TrangorgeOS – gałęź unstable na GitHubie"

[2]: <https://trangorgeos.website/> "Oficjalna strona TrangorgeOS"

[3]: <https://github.com/CTRL-F-org3/Trango>

... (truncated, file is very long)

doc.md

```
# TrangorgeOS Architecture Documentation
```

```
**Generated:** 2026-09-18 21:00:50
```

```
**Files Scanned:** `1709` | **API Items Extracted:** `4008`
```

```
**Languages:** Rust, C/C++, Ada SPARK, Odin, Nim, Assembly
```

```
---
```

Table of Contents

```

- [c.rs](#c.rs)
- [comgrub](#comgrub)
- [comgrub/src](#comgrub-src)
- [comlimine](#comlimine)
- [comlimine/src](#comlimine-src)
- [drivers/amdgpu_driver](#drivers-amdgpu-driver)
- [drivers/audiodriver](#drivers-audiodriver)
- [drivers/intelgpu_driver](#drivers-intelgpu-driver)
- [drivers/netcam_driver](#drivers-netcam-driver)
- [drivers/vgpu](#drivers-vgpu)
- [drivers/wacomgraphic_driver](#drivers-wacomgraphic-driver)
- [driverspace/src](#driverspace-src)
- [driverspace_workspace/crates](#driverspace-workspace-crates)
- [driverspace_workspace/drivers](#driverspace-workspace-drivers)
- [driverspace_workspace/formal](#driverspace-workspace-formal)
- [driverspace_workspace/lib](#driverspace-workspace-lib)
- [driverspacelib/src](#driverspacelib-src)
- [kernel_Workspace/C_base](#kernel-workspace-c-base)
- [kernel_Workspace/Odin_base](#kernel-workspace-odin-base)
- [kernel_Workspace/base](#kernel-workspace-base)
- [kernel_Workspace/core](#kernel-workspace-core)

```

```

- [kernel_workspace/kernel](#kernel-workspace-kernel)
- [kernel_workspace/kernel-bin](#kernel-workspace-kernel-bin)
- [kstd/include](#kstd-include)
- [kstd/src](#kstd-src)
- [libs](#libs)
- [mp4_to_bmp/src](#mp4-to-bmp-src)
- [trangorgelibc/src](#trangorgelibc-src)
- [triang-lang/src](#triang-lang-src)
- [userspace-legacy/demo](#userspace-legacy-demo)
- [userspace-legacy/init](#userspace-legacy-init)
- [userspace-legacy/shell](#userspace-legacy-shell)
- [userspace-legacy/terminal](#userspace-legacy-terminal)

---

## c.rs

<details>
<summary><b>c.rs</b> (2 items)</summary>

#### `STRUCT`: **BlockHeader** <sub>line 57</sub>
```rust
struct BlockHeader* next; /* Wskaźnik do następnego bloku */
```

#### `STRUCT`: **BlockHeader** <sub>line 58</sub>
```rust
struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
```

</details>

## comgrub

<details>
<summary><b>comgrub/build.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```

</details>

## comgrub/src

<details>
<summary><b>comgrub/src/main.rs</b> (2 items)</summary>

#### `FN`: **kernel_main** <sub>line 11</sub>
```rust
fn kernel_main(magic: u32, info: *const u8) -> !;
```

#### `FN`: **panic** <sub>line 15</sub>
```rust
fn panic(_info: &PanicInfo) -> ! {
```

</details>

## comlimine

<details>
<summary><b>comlimine/build.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```

</details>

```

```

## comlimine/src

<details>
<summary><b>comlimine/src/main.c</b> (2 items)</summary>

#### `FUNCTION`: **kernel_main** <sub>line 6</sub>
```c
extern void kernel_main(uint64_t magic, void *info);
```

#### `FUNCTION`: **_start** <sub>line 28</sub>
```c
void _start(void) {
```

</details>

<details>
<summary><b>comlimine/src/main.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {}
```

</details>

## drivers/amdgpu_driver

<details>
<summary><b>drivers/amdgpu_driver/src/main.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```

</details>

## drivers/audiodriver

<details>
<summary><b>drivers/audiodriver/build.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 12</sub>
```rust
fn main() {
```

</details>

<details>
<summary><b>drivers/audiodriver/src/jacklib.rs</b> (6 items)</summary>

#### `STRUCT`: **JackMgr** <sub>line 4</sub>
```rust
pub struct JackMgr {
```

#### `IMPL`: **JackMgr** <sub>line 9</sub>
```rust
impl JackMgr {
```

#### `FN`: **tick** <sub>line 14</sub>
```rust
pub fn tick(&mut self) {
```

#### `FN`: **set_amp** <sub>line 30</sub>
```rust
pub fn set_amp(&mut self, on: bool) {
```

```

`FN`: **present** _{line 35}

```
```rust
pub fn present(&self) -> bool {
 ...
}
```

#### `FN`: \*\*amp\_enabled\*\* <sub>line 40</sub>

```
```rust
pub fn amp_enabled(&self) -> bool {
    ...
}
```

> Czy wzmacniacz (amp) jest aktualnie włączony.

</details>

<details>

<summary>drivers/audiodriver/src/lib.rs (14 items)</summary>

`FN`: **ad_init** _{line 7}

```
```rust
fn ad_init(nam_va: u64, bm_va: u64) -> i32;
...
}
```

#### `FN`: \*\*ad\_play\*\* <sub>line 8</sub>

```
```rust
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
...
}
```

`FN`: **ad_capture** _{line 9}

```
```rust
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
...
}
```

#### `FN`: \*\*ad\_stop\*\* <sub>line 10</sub>

```
```rust
fn ad_stop() -> i32;
...
}
```

`FN`: **ad_jack_present** _{line 11}

```
```rust
fn ad_jack_present() -> i32;
...
}
```

#### `FN`: \*\*ad\_set\_amp\*\* <sub>line 12</sub>

```
```rust
fn ad_set_amp(on: i32) -> i32;
...
}
```

`FN`: **ad_position** _{line 13}

```
```rust
fn ad_position() -> u32;
...
}
```

#### `FN`: \*\*init\*\* <sub>line 16</sub>

```
```rust
pub fn init(nam_va: u64, bm_va: u64) -> bool {
    ...
}
```

`FN`: **play** _{line 20}

```
```rust
pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
 ...
}
```

#### `FN`: \*\*capture\*\* <sub>line 24</sub>

```
```rust
pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    ...
}
```

`FN`: **stop** _{line 28}

```
```rust
pub fn stop() {
 ...
}
```

#### `FN`: \*\*jack\_present\*\* <sub>line 32</sub>

```
```rust
```

```
pub fn jack_present() -> bool {  
    ``
```

```
#### `FN`: **set_amp** <sub>line 36</sub>  
``rust  
pub fn set_amp(on: bool) {  
    ``
```

```
#### `FN`: **position** <sub>line 40</sub>  
``rust  
pub fn position() -> u32 {  
    ``
```

</details>

<details>
<summary>drivers/audiodriver/src/main.rs (1 items)</summary>

```
#### `FN`: **main** <sub>line 8</sub>  
``rust  
fn main() {  
    ``
```

> Punkt wejścia binarki pomocniczej.
>

> Właściwy sterownik działa w driver space przez `ds\_entry` (patrz
> `driverspace/src/main.rs`); ta binarka jest cienkim wrapperem CLI, który
> pozwala sprawdzić stan gniazda jack z poziomu hosta.

</details>

<details>
<summary>drivers/audiodriver/src/odin/driver.odin (8 items)</summary>

```
#### `STRUCT`: **Bdl_Entry** <sub>line 8</sub>  
``odin  
Bdl_Entry :: struct {  
    ``
```

```
#### `PROC`: **nam16** <sub>line 16</sub>  
``odin  
nam16 :: proc(off: u32) -> ^u16 {  
    ``  
    > mixer (NAM) – 16-bit
```

```
#### `PROC`: **bm8** <sub>line 21</sub>  
``odin  
bm8 :: proc(off: u32) -> ^u8 {  
    ``  
    > bus master – 8/16/32
```

```
#### `PROC`: **bm16** <sub>line 25</sub>  
``odin  
bm16 :: proc(off: u32) -> ^u16 {  
    ``
```

```
#### `PROC`: **bm32** <sub>line 29</sub>  
``odin  
bm32 :: proc(off: u32) -> ^u32 {  
    ``
```

```
#### `PROC`: **ad_init** <sub>line 50</sub>  
``odin  
ad_init :: proc "C" (nam_va: u64, bm_va: u64) -> i32 {  
    ``
```

```
#### `PROC`: **ad_stop** <sub>line 73</sub>  
``odin  
ad_stop :: proc "C" () -> i32 {  
    ``
```

```
#### `PROC`: **ad_position** <sub>line 80</sub>  
``odin  
ad_position :: proc "C" () -> u32 {
```

...

</details>

<details>

<summary>drivers/audiodriver/src/odin/jack.odin (2 items)</summary>

`PROC`: **ad_jack_present** _{line 8}

```odin

ad\_jack\_present :: proc "C" () -> i32 {

...

#### `PROC`: \*\*ad\_set\_amp\*\* <sub>line 21</sub>

```odin

ad_set_amp :: proc "C" (on: i32) -> i32 {

...

</details>

<details>

<summary>drivers/audiodriver/src/odin/microphone.odin (1 items)</summary>

`PROC`: **ad_capture** _{line 6}

```odin

ad\_capture :: proc "C" (data\_phys: u64, len: u32, bdl\_va: u64, bdl\_phys: u64) -> i32 {

...

</details>

<details>

<summary><b>drivers/audiodriver/src/odin/speaker.odin</b> (1 items)</summary>

#### `PROC`: \*\*ad\_play\*\* <sub>line 6</sub>

```odin

ad_play :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {

...

</details>

<details>

<summary>drivers/audiodriver/src/tone.rs (1 items)</summary>

`FN`: **fill_square** _{line 1}

```rust

pub fn fill\_square(buf: &mut [u8], periods: u32) {

...

</details>

## drivers/intelgpu\_driver

<details>

<summary><b>drivers/intelgpu\_driver/src/main.rs</b> (1 items)</summary>

#### `FN`: \*\*main\*\* <sub>line 1</sub>

```rust

fn main() {

...

</details>

drivers/netcam_driver

<details>

<summary>drivers/netcam_driver/src/main.rs (1 items)</summary>

`FN`: **main** _{line 1}

```rust

fn main() {

...

</details>

## drivers/vgpu

<details>

<summary><b>drivers/vgpu/vgpu.h</b> (7 items)</summary>

#### `FUNCTION`: **\*\*vgpu\_init\*\*** <sub>line 22</sub>

```c

bool vgpu_init(uint32_t w, uint32_t h);

```

#### `FUNCTION`: **\*\*vgpu\_get\_info\*\*** <sub>line 23</sub>

```c

vgpu_info_t vgpu_get_info(void);

```

#### `FUNCTION`: **\*\*vgpu\_clear\*\*** <sub>line 25</sub>

```c

void vgpu_clear(uint32_t color);

```

#### `FUNCTION`: **\*\*vgpu\_pixel\*\*** <sub>line 26</sub>

```c

void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

```

#### `FUNCTION`: **\*\*vgpu\_surface\_create\*\*** <sub>line 28</sub>

```c

int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);

```

#### `FUNCTION`: **\*\*vgpu\_surface\_present\*\*** <sub>line 29</sub>

```c

bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

```

#### `FUNCTION`: **\*\*vgpu\_process\_ring\*\*** <sub>line 31</sub>

```c

void vgpu_process_ring(volatile void *ring);

```

</details>

<details>

<summary><b>drivers/vgpu/vgpu\_driver.c</b> (11 items)</summary>

#### `FUNCTION`: **\*\*pci\_void\*\*** <sub>line 12</sub>

```c

static void pci_void(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)

```

#### `FUNCTION`: **\*\*pci\_val\*\*** <sub>line 20</sub>

```c

static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)

```

#### `FUNCTION`: **\*\*vbe\_write\*\*** <sub>line 33</sub>

```c

static void vbe_write(uint16_t idx, uint16_t val)

```

#### `FUNCTION`: **\*\*vgpu\_init\*\*** <sub>line 40</sub>

```c

bool vgpu_init(uint32_t w, uint32_t h)

```

#### `FUNCTION`: **\*\*vgpu\_get\_info\*\*** <sub>line 98</sub>

```c

vgpu_info_t vgpu_get_info(void)

```

#### `FUNCTION`: **\*\*vgpu\_pixel\*\*** <sub>line 104</sub>

```c

void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color)

```

    ...

#### `FUNCTION`: **vgpu_clear** <sub>line 113</sub>
```c
void vgpu_clear(uint32_t color)
```

#### `FUNCTION`: **vgpu_surface_create** <sub>line 128</sub>
```c
int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out)
```

#### `FUNCTION`: **vgpu_surface_present** <sub>line 168</sub>
```c
bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y)
```

#### `FUNCTION`: **vgpu_process_ring** <sub>line 190</sub>
```c
void vgpu_process_ring(volatile vgpu_slot_t *ring)
```

#### `FUNCTION`: **ds_entry** <sub>line 244</sub>
```c
void ds_entry(uint64_t params_va)
```

</details>

## drivers/wacomgraphic_driver

<details>
<summary><b>drivers/wacomgraphic_driver/src/main.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
 ...
}
```

</details>

## driverspace/src

<details>
<summary><b>driverspace/src/drivers/storage.rs</b> (4 items)</summary>

#### `STRUCT`: **StorageDrv** <sub>line 9</sub>
```rust
pub struct StorageDrv {
 ...
}
> Storage driver stub. It only satisfies the driver registry so that
> driver space can boot; a real disk driver can slot in later.

`IMPL`: **StorageDrv** _{line 13}
```rust
impl StorageDrv {
    ...
}

#### `IMPL`: **Driver** <sub>line 19</sub>
```rust
impl Driver for StorageDrv {
 ...
}

`FN`: **init** _{line 20}
```rust
fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
    ...
}
```

</details>

<details>
<summary>driverspace/src/main.rs (1 items)</summary>

```

```
`FN`: **panic** _{line 32}
```rust
fn panic(_info: &core::panic::PanicInfo) -> ! {
```
```

</details>

## driverspace\_workspace/crates

```
<details>
<summary>driverspace_workspace/crates/ds-manager/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

## driverspace\_workspace/drivers

```
<details>
<summary>driverspace_workspace/drivers/amdgpu-driver/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```
<details>
<summary>driverspace_workspace/drivers/audiodriver/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```
<details>
<summary>driverspace_workspace/drivers/intelgpu-driver/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```
<details>
<summary>driverspace_workspace/drivers/netcam_driver/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```
<details>
<summary>driverspace_workspace/drivers/vgpu/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```
<details>
<summary>driverspace_workspace/drivers/wacomgraphic_driver/src/main.rs (1 items)</summary>
```

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

## driverspace\_workspace/formal

<details>

<summary><b>driverspace\_workspace/formal/ds-spark-core/src/ds\_buddy\_math.adb</b> (7 items)</summary>

```
`PACKAGE`: **body** _{line 2}
```ada
package body DS_Buddy_Math with
```
```

> formal/ds-spark-core/src/ds\_buddy\_math.adb

```
`FUNCTION`: **Power_Of_Two** _{line 6}
```ada
function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is
```
```

```
`FUNCTION`: **Is_Aligned** _{line 11}
```ada
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
```
```

```
`FUNCTION`: **Get_Buddy_Address** _{line 18}
```ada
function Get_Buddy_Address (
```
```

```
`FUNCTION`: **Get_Left_Child_Address** _{line 31}
```ada
function Get_Left_Child_Address (
```
```

```
`FUNCTION`: **Get_Right_Child_Address** _{line 40}
```ada
function Get_Right_Child_Address (
```
```

```
`FUNCTION`: **Safe_Calculate_DMA_End** _{line 50}
```ada
function Safe_Calculate_DMA_End (
```
```

</details>

<details>

<summary><b>driverspace\_workspace/formal/ds-spark-core/src/ds\_buddy\_math.ads</b> (9 items)</summary>

```
`PACKAGE`: **DS_Buddy_Math** _{line 4}
```ada
package DS_Buddy_Math with
```
```

```
`TYPE`: **Phys_Addr** _{line 9}
```ada
type Phys_Addr is new Unsigned_64;
```
> Typy bazowe dla fizycznej pamieci (bare-metal)
```

```
`TYPE`: **Block_Size** _{line 10}
```ada
type Block_Size is new Unsigned_64;
```
```

```
`FUNCTION`: **Power_Of_Two** _{line 17}
```ada
function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
```
```

> Funkcja pomocnicza: Obliczanie  $2^N$  (Shift left)

```
`FUNCTION`: **Is_Aligned** _{line 23}
```ada
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
```
```

> Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku

```
`FUNCTION`: **Get_Buddy_Address** _{line 32}
```ada
function Get_Buddy_Address (
```
```

```
`FUNCTION`: **Get_Left_Child_Address** _{line 47}
```ada
function Get_Left_Child_Address (
```
```

```
`FUNCTION`: **Get_Right_Child_Address** _{line 55}
```ada
function Get_Right_Child_Address (
```
```

```
`FUNCTION`: **Safe_Calculate_DMA_End** _{line 69}
```ada
function Safe_Calculate_DMA_End (
```
```

</details>

## driverspace\_workspace/lib

<details>

<summary><b>driverspace\_workspace/lib/ds-formal-ffi/build.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 10}
```rust
fn main() {
```
```

</details>

<details>

<summary><b>driverspace\_workspace/lib/ds-formal-ffi/src/lib.rs</b> (6 items)</summary>

```
`FN`: **ds_spark_safe_add_32** _{line 7}
```rust
pub fn ds_spark_safe_add_32(a: u32, b: u32) -> u32;
```
```

```
`FN`: **ds_spark_safe_array_index** _{line 10}
```rust
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
```
```

```
`FN`: **safe_add** _{line 21}
```rust
pub fn safe_add(a: u32, b: u32) -> u32 {
```
```

```
`FN`: **ds_spark_buddy_get_buddy** _{line 28}
```rust
pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
```
```

```
`FN`: **ds_spark_safe_dma_end** _{line 29}
```rust
pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
```
```

```
`FN`: **get_buddy** _{line 35}
```rust
pub fn get_buddy(addr: u64, exp: u64) -> u64 {
```
```

...

</details>

<details>

<summary><b>driverspace\_workspace/lib/ds-fw-audio/src/lib.rs</b> (2 items)</summary>

#### `FN`: **\*\*add\*\*** <sub>line 1</sub>

```rust

pub fn add(left: u64, right: u64) -> u64 {

...

`FN`: ****it_works**** _{line 10}

```rust

fn it\_works() {

...

</details>

<details>

<summary><b>driverspace\_workspace/lib/ds-fw-block/src/request.rs</b> (8 items)</summary>

#### `ENUM`: **\*\*RequestState\*\*** <sub>line 7</sub>

```rust

pub enum RequestState {

...

`STRUCT`: ****IoRequest**** _{line 15}

```rust

pub struct IoRequest {

...

> High-level I/O request with metadata.

#### `IMPL`: **\*\*IoRequest\*\*** <sub>line 27</sub>

```rust

impl IoRequest {

...

`FN`: ****new**** _{line 28}

```rust

pub fn new(id: u64, lba: u64, block\_count: u32, buffer: \*mut u8, len: usize, is\_write: bool) -> Self {

...

#### `FN`: **\*\*state\*\*** <sub>line 42</sub>

```rust

pub fn state(&self) -> RequestState {

...

`FN`: ****mark_in_progress**** _{line 52}

```rust

pub fn mark\_in\_progress(&self) {

...

#### `FN`: **\*\*mark\_completed\*\*** <sub>line 56</sub>

```rust

pub fn mark_completed(&self) {

...

`FN`: ****mark_failed**** _{line 63}

```rust

pub fn mark\_failed(&self) {

...

</details>

<details>

<summary><b>driverspace\_workspace/lib/ds-fw-block/src/traits.rs</b> (11 items)</summary>

#### `STRUCT`: **\*\*BlockRequest\*\*** <sub>line 6</sub>

```rust

pub struct BlockRequest {

...

> Represents a single I/O request to a block device.

```

#### `TRAIT`: **BlockDevice** <sub>line 16</sub>
```rust
pub trait BlockDevice {
 <sub>
 > Core trait for all block devices.
 > Drivers must implement this to integrate with the block subsystem.
 </sub>

 #### `FN`: **read_blocks** _{line 18}
    ```rust
    fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;
    <sub>
    > Read blocks from the device starting at LBA.
    </sub>

    #### `FN`: **write_blocks** <sub>line 21</sub>
    ```rust
 fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;
 <sub>
 > Write blocks to the device starting at LBA.
 </sub>

 #### `FN`: **flush** _{line 24}
    ```rust
    fn flush(&mut self) -> Result<(), DsError>;
    <sub>
    > Flush internal caches to physical media.
    </sub>

    #### `FN`: **geometry** <sub>line 27</sub>
    ```rust
 fn geometry(&self) -> BlockGeometry;
 <sub>
 > Get device geometry (block size, total blocks).
 </sub>

 #### `FN`: **supports_trim** _{line 30}
    ```rust
    fn supports_trim(&self) -> bool { false }
    <sub>
    > Check if device supports TRIM/discard.
    </sub>

    #### `FN`: **submit_request** <sub>line 33</sub>
    ```rust
 fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
 <sub>
 > Optional: Submit async request (for advanced schedulers).
 </sub>

 #### `STRUCT`: **BlockGeometry** _{line 40}
    ```rust
    pub struct BlockGeometry {
    <sub>

    #### `IMPL`: **BlockGeometry** <sub>line 46</sub>
    ```rust
 impl BlockGeometry {

 #### `FN`: **total_bytes** _{line 47}
    ```rust
    pub fn total_bytes(&self) -> u64 {

</details>

<details>
<summary><b>driverspace_workspace/lib/ds-fw-gpu/src/lib.rs</b> (2 items)</summary>

#### `FN`: **add** <sub>line 1</sub>
```rust
pub fn add(left: u64, right: u64) -> u64 {
 <sub>

`FN`: **it_works** _{line 10}
```rust
fn it_works() {
    <sub>

```

```

</details>

<details>
<summary><b>driverspace_workspace/lib/ds-fw-input/src/lib.rs</b> (2 items)</summary>

#### `FN`: **add** <sub>line 1</sub>
```rust
pub fn add(left: u64, right: u64) -> u64 {
```

#### `FN`: **it_works** <sub>line 10</sub>
```rust
fn it_works() {
```

</details>

<details>
<summary><b>driverspace_workspace/lib/ds-ipc/src/client.rs</b> (4 items)</summary>

#### `STRUCT`: **ManagerClient** <sub>line 7</sub>
```rust
pub struct ManagerClient {
```

#### `IMPL`: **ManagerClient** <sub>line 11</sub>
```rust
impl ManagerClient {
```

#### `FN`: **request_mmio** <sub>line 16</sub>
```rust
pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
```

#### `FN`: **bind_irq** <sub>line 41</sub>
```rust
pub fn bin

... (truncated, file is very long)

```

## driverspace\_workspace/TODO.md

# TODO.md: Driver Space Migration and Implementation Guide

### ## Overview

This document outlines the pending tasks, architectural constraints, and execution steps for migrating and

The legacy monolithic implementations (`driverspacelib` and the old `driverspace` daemon) are deprecated.

### ## 1. Prerequisites

Before beginning driver implementation, the foundational libraries located in `lib/` must be fully implemen

- [ ] **`lib/kapi-abi`**: Finalize all IPC message structures, opcodes, and error codes. Ensure `cbindgen
- [ ] **`lib/kapi-syscall`**: Implement architecture-specific SVC/syscall wrappers.
- [ ] **`lib/ds-ipc`**: Stabilize the channel, port, and shared memory abstractions.
- [ ] **`lib/ds-mem`**: Finalize the `no\_std` allocators (Buddy/Slab) and DMA mapping utilities.
- [ ] **`lib/ds-fw`**: Define the core traits for device classes (`BlockDevice`, `AudioDevice`, `Displa

### ## 2. Driver Implementation Checklist

The following drivers require migration from the legacy codebase or complete reimplementatio

#### ### 2.1. Graphics and Display

- [ ] **`amdgpu\_driver`**
  - [ ] Implement MMIO register read/write wrappers.
  - [ ] Implement ring buffer management for command submission.
  - [ ] Implement Display PHY initialization.
  - [ ] Implement the `DisplayDevice` and `GpuDevice` traits from `ds-fw-gpu`.
- [ ] **`intelgpu\_driver`**
  - [ ] Implement Graphics Technology (GT) and Display Engine (DE) initialization.
  - [ ] Implement GuC (Graphics Microcontroller) firmware loading.
  - [ ] Implement the `DisplayDevice` trait from `ds-fw-gpu`.
- [ ] **`vgpu` (Virtual GPU)**

- [ ] Set up the Rust-to-C FFI bridge (`build.rs` and `ffi.rs`).
- [ ] Implement virtual framebuffer memory management.
- [ ] Ensure the C implementation strictly uses the `kapi-abi` headers.

### ### 2.2. Audio

- [ ] **`audiodriver`**
  - [ ] Implement DMA stream management for audio buffers.
  - [ ] Implement codec communication protocols (I2C / Intel HDA).
  - [ ] Integrate the Odin/C DSP components via the defined FFI boundaries.
  - [ ] Implement the `AudioDevice` trait from `ds-fw-audio`.

### ### 2.3. Input and HID

- [ ] **`wacomgraphic\_driver`**
  - [ ] Implement Wacom-specific HID report parsing.
  - [ ] Implement raw-to-logical data conversion for pen pressure and tilt (X/Y).
  - [ ] Implement the `InputDevice` trait from `ds-fw-input`.

### ### 2.4. Networking and Multimedia

- [ ] **`netcam\_driver`**
  - [ ] Implement USB Video Class (UVC) protocol parsing.
  - [ ] Implement isochronous USB transfer handling.
  - [ ] Implement device control interfaces (brightness, contrast, zoom).

## ## 3. Strict Architectural Rules for Contributors

All contributors must adhere to the following rules when writing driver code. Violations will result in r

1. **Unidirectional Dependencies**:
  - Drivers (`drivers/`) may **only** depend on libraries in `lib/`.
  - Drivers must **never** depend on the management daemons in `crates/`. Communication with the manager
2. **No Standard Library**:
  - All Rust driver code must be `#![no\_std]`. The use of the Rust standard library is strictly prohibi
3. **Zero Code Duplication**:
  - If a utility, protocol parser, or hardware abstraction is needed by more than one driver, it must be
4. **Strict FFI Boundaries**:
  - If a driver requires C or Odin code, it must be isolated in a dedicated subdirectory (e.g., `c/` or
  - Compilation of foreign code must be handled via `build.rs` using the `cc` crate or equivalent toolin
  - Rust wrappers for foreign functions must be isolated in an `ffi.rs` file.
5. **Capability and Resource Management**:
  - Drivers must not attempt to access hardware resources (MMIO, IRQs, DMA) directly via hardcoded addre

## ## 4. Execution Steps for a New Driver

When starting work on a driver, follow this exact sequence:

1. **Initialize the Crate**: Create the directory under `drivers/<driver\_name>/` and generate a new `Cargo
2. **Update Workspace**: Add the new crate path to the `members` array in the root `driverspace\_workspace
3. **Define Dependencies**: Add only the necessary `lib/` crates to the driver's `Cargo.toml` (e.g., `kap
4. **Implement Entry Point**: Create `main.rs` (or `lib.rs`), initialize the driver runtime, and establis
5. **Implement the Trait**: Implement the required trait from the corresponding `ds-fw-\*` framework.
6. **Hardware Logic**: Implement the actual hardware register manipulation and state machines.

## ## 5. Current Status

- [ ] Workspace structure created and validated.
- [ ] Legacy `driverspacelib` code analyzed for extraction.
- [ ] `lib/kapi-abi` definitions finalized.
- [ ] First driver (`amdgpu\_driver` or `audiodriver`) migrated to the new workspace.

\*For questions regarding the architecture or the new Kernel API, refer to the main Architecture Documenta

## driverspace\_workspace/Tree\_readme.md

driverspace_workspace/	
■■■■ Cargo.toml	# Główny workspace manifest
■■■■ rust-toolchain.toml	# Wymuszenie nightly i specyficznych targetów
■	
■■■■ lib/	# ■ FUNDAMENT: API, INFRASTRUKTURA I FRAMEWORKI
■ ■	
■ ■■■■ kapi-abi/	# Nowe API Jędra: Definicje binarne (Single Source of Truth)
■ ■ ■■■ Cargo.toml	
■ ■ ■■■ build.rs	# Generowanie nagłówek C/Odin przez cbindgen
■ ■ ■■■ src/	
■ ■ ■■■ lib.rs	# Tylko re-eksporty i flagi no_std
■ ■ ■■■ primitives.rs	# Podstawowe typy (Handle, CapId, Status)
■ ■ ■■■ opcodes.rs	# Enumy z numerami wywołań (Syscall/IPC opcodes)

```

■ ■ ■■■ errors.rs # Wspólne kody błędów (DsError)
■ ■ ■■■ capabilities.rs # Struktury uprawnień (Capability tokens)
■ ■ ■■■ wire/ # Serializacja/Deserializacja wiadomości
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ encode.rs # Zapis do bufora
■ ■ ■ ■■■ decode.rs # Odczyt z bufora
■ ■ ■ ■■■ endian.rs # Konwersje kolejności bajtów
■ ■ ■■■ payloads/ # Konkretnie struktury wiadomości ([repr(C)])
■ ■ ■ ■■■ mod.rs
■ ■ ■ ■■■ mem.rs # ■■■dania mapowania pamięci
■ ■ ■ ■■■ ipc.rs # ■■■dania tworzenia kanałów
■ ■ ■ ■■■ dev.rs # ■■■dania rejestracji urządzeń
■ ■ ■ ■■■ irq.rs # ■■■dania obsługi przerwań
■ ■
■ ■■■ kapi-syscall/ # Surowe wywołania systemowe (SVC / INT)
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■ ■■■ lib.rs
■ ■ ■ ■■■ arch/
■ ■ ■ ■ ■■■ mod.rs
■ ■ ■ ■ ■■■ x86_64.rs # Makra asm! dla x86_64 (syscall/sysenter)
■ ■ ■ ■ ■■■ riscv64.rs # Makra asm! dla RISC-V (ecall)
■ ■ ■ ■ ■■■ wrappers.rs # Bezpieczne (lub unsafe) wrappery nad surowym asm
■ ■
■ ■■■ ds-ipc/ # Infrastruktura IPC (Komunikacja między procesami)
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■ ■■■ lib.rs
■ ■ ■ ■■■ channel.rs # Abstrakcja kanału (dwukierunkowego)
■ ■ ■ ■■■ port.rs # Abstrakcja portu (nasłuchiwanie)
■ ■ ■ ■■■ router.rs # Logika routowania wiadomości
■ ■ ■ ■■■ shared_mem.rs # Zarządzanie buforami pamięci współdzielonej
■ ■ ■ ■■■ sync/ # Primitives synchronizacji dla IPC
■ ■ ■ ■ ■■■ mod.rs
■ ■ ■ ■ ■■■ spinlock.rs # Spinlock (no_std)
■ ■ ■ ■ ■■■ semaphore.rs # Semafore dla kolejek wiadomości
■ ■
■ ■■■ ds-mem/ # Zarządzanie pamięcią w przestrzeni sterownika
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■ ■■■ lib.rs
■ ■ ■ ■■■ alloc/
■ ■ ■ ■ ■■■ mod.rs
■ ■ ■ ■ ■■■ buddy.rs # Alokator Buddy (dla dużych bloków)
■ ■ ■ ■ ■■■ slab.rs # Alokator Slab (dla małych, stałych obiektów)
■ ■ ■ ■ ■■■ heap.rs # Globalny heap sterownika
■ ■ ■ ■ ■■■ dma/ # Obsługa DMA (Direct Memory Access)
■ ■ ■ ■ ■ ■■■ mod.rs
■ ■ ■ ■ ■ ■■■ buffer.rs # Bufory DMA (pamięć fizyczna/ciągła)
■ ■ ■ ■ ■ ■■■ mapping.rs # Mapowanie pamięci dla urządzeń
■ ■ ■ ■ ■ ■■■ vmm.rs # Abstrakcje nad Wirtualną Pamięcią (stronicowanie)
■ ■
■ ■■■ ds-log/ # System logowania
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■ ■■■ lib.rs
■ ■ ■ ■■■ levels.rs # Enumy poziomów (Trace, Info, Error, Panic)
■ ■ ■ ■■■ formatter.rs # Formatowanie tekstu (bez std::fmt)
■ ■ ■ ■■■ transport.rs # Wysyłanie logów przez IPC do menedżera
■ ■ ■ ■■■ macros.rs # Makra: ds_log::info!, ds_log::error!
■ ■
■ ■■■ ds-fw-block/ # Framework dla urządzeń blokowych (Dyski, USB Mass)
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■ ■■■ lib.rs
■ ■ ■ ■■■ traits.rs # Trait BlockDevice (read, write, flush)
■ ■ ■ ■■■ request.rs # Struktura ■■■dania I/O (LBA, rozmiar, bufor)
■ ■ ■ ■■■ queue.rs # Kolejka ■■■da (I/O Scheduler)
■ ■ ■ ■■■ partition.rs # Abstrakcja tablic partycji (MBR/GPT)
■ ■
■ ■■■ ds-fw-audio/ # Framework dla Audio
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/

```

```

■ ■ ■■■ lib.rs
■ ■ ■■■ traits.rs # Trait AudioDevice (play, record)
■ ■ ■■■ stream.rs # Abstrakcja strumienia audio
■ ■ ■■■ formats.rs # Formaty próbek (PCM, Float, Sample rates)
■ ■ ■■■ ringbuffer.rs # Lock-free ring buffer dla danych audio
■ ■
■ ■■■ ds-fw-gpu/ # Framework dla GPU / Display
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■■■ lib.rs
■ ■ ■■■ traits.rs # Trait DisplayDevice, GpuDevice
■ ■ ■■■ framebuffer.rs # Abstrakcja bufora ramki (pitch, format)
■ ■ ■■■ modeset.rs # Ustawianie rozdzielczości i trybu
■ ■ ■■■ edid.rs # Parser EDID (dane monitora)
■ ■ ■■■ cmd_buffer.rs # Abstrakcja bufora poleceń GPU (Command Submission)
■ ■
■ ■■■ ds-fw-input/ # Framework dla HID (Klawiatura, Mysz, Wacom)
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■■■ lib.rs
■ ■ ■■■ traits.rs # Trait InputDevice (poll_event)
■ ■ ■■■ events.rs # Typy zdarzeń (KeyPress, MouseMove, PenPressure)
■ ■ ■■■ scancodes.rs # Mapowanie scancode'ów na kody klawiszy
■ ■ ■■■ hid_parser.rs # Parser raportów HID (USB HID descriptors)
■ ■
■ ■■■ crates/ # ■ ZARZĄDZANIE: DAEMONY DRIVERSPACE
■ ■
■ ■■■ ds-manager/ # Główny daemon zarządzający sterownikami
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■■■ main.rs # Entry point, inicjalizacja obsługi zdarzeń
■ ■ ■■■ lifecycle/ # Zarządzanie cyklem życia sterowników
■ ■ ■■■ mod.rs
■ ■ ■■■ spawn.rs # Ładowanie i startowanie procesów sterowników
■ ■ ■■■ monitor.rs # Watchdog (restartowanie crashy sterowników)
■ ■ ■■■ shutdown.rs # Bezpieczne zamykanie
■ ■ ■■■ resources/ # Przydzielanie zasobów sprzętowych
■ ■ ■■■ mod.rs
■ ■ ■■■ irq.rs # Routing i przydzielanie IRQ
■ ■ ■■■ mmio.rs # Mapowanie pamięci urządzeń
■ ■ ■■■ dma.rs # Przydzielanie buforów DMA
■ ■ ■■■ ipc_handler.rs # Nasłuchiwanie danych od jądra i sterowników
■ ■ ■■■ config.rs # Wczytywanie konfiguracji (np. z TOML/JSON)
■ ■
■ ■■■ ds-registry/ # Rejestr urządzeń i sterowników
■ ■ ■■■ Cargo.toml
■ ■ ■■■ src/
■ ■ ■■■ lib.rs
■ ■ ■■■ database.rs # Przechowywanie mapy DeviceID -> Driver
■ ■ ■■■ matching.rs # Logika dopasowywania (PCI ID, USB ID, ACPI)
■ ■ ■■■ state.rs # Śledzenie stanu urządzeń (Podłączzone, Usunięte, Błędne)
■ ■
■ ■■■ drivers/ # ■ STEROWNIKI SPRZĘTOWE
■ ■
■ ■■■ amdgpu_driver/ # Sterownik AMD GPU
■ ■ ■■■ Cargo.toml
■ ■ ■■■ build.rs # (Opcjonalnie) kompilacja C jeśli używasz blobów
■ ■ ■■■ src/
■ ■ ■■■ main.rs # Entry point, inicjalizacja ds-runtime
■ ■ ■■■ probe.rs # Detekcja karty, odczyt BAR-ów
■ ■ ■■■ mmio.rs # Wrappery do odczytu/zapisu rejestrów MMIO
■ ■ ■■■ irq.rs # Obsługa przerwań GPU
■ ■ ■■■ registers/ # Definicje rejestrów (atomowo!)
■ ■ ■■■ mod.rs
■ ■ ■■■ gc.rs # Graphics Controller
■ ■ ■■■ sdma.rs # System DMA
■ ■ ■■■ nbio.rs # Northbridge IO
■ ■ ■■■ ring_buffer.rs # Implementacja pierścieni poleceń (Ring Buffers)
■ ■ ■■■ display/ # Podmoduł wyświetlania
■ ■ ■■■ mod.rs
■ ■ ■■■ phy.rs # Inicjalizacja PHY (Physical layer)
■ ■
■ ■■■ audiodriver/ # Sterownik Audio (Hybryda Rust/C/Okładka na Odina)

```

```

■ ■■■ Cargo.toml
■ ■■■ build.rs # Kompilacja kodu C i linkowanie z Odinem
■ ■■■ c/ # Kod C (jeżeli potrzebny do specyficznych DSP)
■ ■ ■■■ dsp.c
■ ■ ■■■ dsp.h
■ ■■■ src/
■ ■■■ main.rs
■ ■■■ codec.rs # Komunikacja z kodekiem audio (I2C/HDA)
■ ■■■ dma_stream.rs # Zarządzanie strumieniami DMA dla audio
■ ■■■ ffi.rs # Funkcje `extern "C"` wywoływane przez C/Okładki
■
■■■ intelgpu_driver/ # Sterownik Intel GPU (i915/xe)
■ ■■■ Cargo.toml
■ ■■■ src/
■ ■■■ main.rs
■ ■■■ probe.rs
■ ■■■ gt.rs # Graphics Technology (rdzeń GPU)
■ ■■■ display.rs # Display Engine (DE)
■ ■■■ guc.rs # GuC (Graphics Microcontroller) firmware loading
■
■■■ netcam_driver/ # Sterownik kamer sieciowych (UVC/USB)
■ ■■■ Cargo.toml
■ ■■■ src/
■ ■■■ main.rs
■ ■■■ uvc.rs # USB Video Class protocol parser
■ ■■■ isoc.rs # Obsługa transferów izochronicznych USB
■ ■■■ controls.rs # Panel sterowania (jasność, kontrast, zoom)
■
■■■ vgpu/ # Wirtualne GPU (Głównie C, z Rust FFI)
■ ■■■ Cargo.toml
■ ■■■ build.rs # Kompilacja vgpu_driver.c
■ ■■■ c/
■ ■■■ vgpu_driver.c # Główna logika w C
■ ■■■ vgpu.h
■ ■■■ vgpu_mmio.c # Operacje na pamięci w C
■ ■■■ src/
■ ■■■ lib.rs # Rust wrapper, eksportujący ABI dla ds-manager
■ ■■■ ffi.rs # Definicje `extern "C"` dla funkcji z C
■
■■■ wacomgraphic_driver/ # Tablet graficzny Wacom (HID)
■ ■■■ Cargo.toml
■ ■■■ src/
■ ■■■ main.rs
■ ■■■ hid_report.rs # Parsowanie specyficznych raportów Wacom
■ ■■■ pressure.rs # Konwersja surowych danych na nacisk
■ ■■■ tilt.rs # Obsługa nachylenia rysika (Tilt X/Y)

```

## driverspace\_workspace/Workspace.md

Oto profesjonalna dokumentacja architektury, przygotowana w języku angielskim, zgodnie z Twoimi wytycznymi

\*\*\*

# TrangorgeOS Driver Space Architecture and Workspace Documentation

## 1. Executive Summary

This document outlines the architectural redesign of the Driver Space subsystem within TrangorgeOS. The c

## 2. Workspace Topology

The `driverspace\_workspace` is divided into three primary pillars. This strict separation of concerns dic

```text

```

driverspace_workspace/
■■■ Cargo.toml          # Workspace root configuration
■■■ lib/                # Core libraries, frameworks, and the new Kernel API
■■■ crates/             # Driver space management daemons and services
■■■ drivers/            # Individual hardware driver implementations
```

```

## 3. Component Breakdown

### 3.1. `lib/` (Core Libraries and Kernel API)

The `lib/` directory serves as the foundational layer of the driver space. It contains the new Kernel API

```

* **Kernel API (`kapi-`)**
* `kapi-abi`: The single source of truth for all data structures, IPC message formats, opcodes, and
* `kapi-syscall`: Low-level, architecture-specific wrappers for kernel transitions (e.g., SVC calls)
* **Infrastructure (`ds-`)**
* `ds-ipc`: Inter-process communication primitives, channel management, and message routing logic.
* `ds-mem`: Memory management utilities, including custom allocators for `no_std` environments, DMA
* `ds-log`: Standardized logging infrastructure that routes driver logs to the central manager or kernel
* **Device Frameworks (`ds-fw-`)**
* `ds-fw-block`, `ds-fw-audio`, `ds-fw-gpu`, `ds-fw-input`: High-level abstractions for specific device

```

### 3.2. `crates/` (Driver Space Management)

The `crates/` directory contains the privileged user-space (or kernel-space) daemons responsible for orchestrating the driver space.

```

* `ds-manager`: The central daemon responsible for device enumeration, resource allocation (IRQs, MMIO)
* `ds-registry`: A database service that maintains the mapping of hardware device identifiers (e.g., PCI IDs)

```

### 3.3. `drivers/` (Hardware Drivers)

The `drivers/` directory contains the actual implementations for specific hardware peripherals (e.g., `armv8`, `x86\_64`).

```

* Each driver is an independent crate (or an external project for C/Odin implementations).
* Drivers implement the traits defined in the `ds-fw-*` frameworks.
* Drivers communicate with the `ds-manager` and the kernel exclusively through the APIs provided in `libkapi`.

```

## 4. The New Kernel API (`kapi`)

The legacy C headers (`libs/\*.h`) and the old `abi.rs` are deprecated. The new `kapi` (Kernel API) is designed to be a single source of truth for all kernel interfaces.

Key characteristics of the new API:

- Explicit Message Passing:** All communication between the kernel, the manager, and the drivers is handled through explicit channels.
- Capability-Based Security:** Resource sharing (like memory or IRQs) is handled through explicit capabilities.
- Language Agnostic:** The ABI is designed to be easily consumable by Rust, C, and Odin, facilitating cross-language interoperability.

## 5. Architectural Rules and Dependency Graph

To maintain the integrity of the workspace, the following strict rules apply:

- Unidirectional Dependencies:**
  - `drivers/` may only depend on `lib/`.
  - `crates/` may only depend on `lib/`.
  - Crucial Rule:** `drivers/` must **never** depend on `crates/`. Drivers must not have compile-time dependencies on other drivers.
- No Code Duplication:** If a utility, protocol parser, or hardware abstraction is needed by more than one driver, it must be shared in the `lib/` directory.
- Strict `no\_std` Compliance:** All crates within `lib/` and `crates/` must be `#![no\_std]`. Standard library features are provided via the `libkapi` crate.

## 6. Cross-Language Interoperability (FFI)

TrangorgeOS utilizes multiple languages (Rust, C, Odin, Nim). The `lib/kapi-abi` crate acts as the central interface for all languages.

```

* Rust structures must use `#[repr(C)]`.
* A build script (`build.rs`) utilizing `cbindgen` should be implemented in `kapi-abi` to automatically generate C headers.

```

## 7. Summary

This modular workspace architecture transforms the TrangorgeOS driver space from a tightly coupled monolith into a flexible, maintainable, and secure ecosystem.

**[driverspace\\_workspace/formal/README.md](#)**

**[kernel\\_Workspace/C\\_base/kc-abi/docs/abi\\_spec.md](#)**

**[kernel\\_Workspace/C\\_base/kc-abi/docs/architecture.md](#)**

**[kernel\\_Workspace/C\\_base/kc-abi/docs/memory\\_model.md](#)**

**[kernel\\_Workspace/C\\_base/kc-abi/kc-abi/docs/abi\\_spec.md](#)**

**[kernel\\_Workspace/C\\_base/kc-abi/kc-abi/docs/architecture.md](#)**

**kernel\_Workspace/README.md**



**kernel\_Workspace/kernel/README.md**

kernel\_Workspace/kernel/src/mm/MM\_PLAN\_ULEPSZENIA.md

```
P0.1. Blokada wywołująca przerwanie nie są blokadami SMP
```

PMM, slab i przestrzenie adresowe używają wzorca `pushfq; cli` oraz lokalnego licznika zagnieżdżenia. Wy

**\*\*Pliki:\*\*** `kernel/src/mm/alloc/physical/pmm.c`, `kernel/src/mm/alloc/heap/slab.c`, `kernel/src/mm/process

**\*\*Zadanie:\*\*** zastąpi blokady per-CPU rzeczywistymi spinlockami ticket/MCS albo istniejącym prymitywem S

**\*\*Kryterium akceptacji:\*\*** test wielordzeniowy wykonuje równoległe co najmniej 100 000 alokacji i zwolnień

#### P0.2. Double-free w slabie może uszkodzić listę wolnych obiektów

`slab\_free()` sprawdza zakres i wyrównanie wskaźnika, ale nie przechowuje informacji, czy konkretny obiekt

**\*\*Plik:\*\*** `kernel/src/mm/alloc/heap/slab.c`, szczególnie funkcja `slab\_free()`.

**\*\*Kryterium akceptacji:\*\*** drugie `slab\_free(ptr)` nie modyfikuje allocatorów i generuje błąd diagnostyczny

#### P0.3. Brak walidacji overflow w API przestrzeni adresowych

W wielu miejscach granice są liczone jako `addr + len`, a następnie wyrównywane, np. w `aspace\_map\_at`,

**\*\*Plik:\*\*** `kernel/src/mm/process/address\_space.c`, okolice obliczeń `a`/`b`.

**\*\*Zadanie:\*\*** wprowadzić wspólną funkcję `range\_from\_addr\_len(addr, len, alignment, out\_start, out\_end)`,

1. odrzuca `len == 0`;
2. sprawdza `addr + len` bez overflow;
3. bezpiecznie wyrównuje początek w dół i koniec w górę;
4. sprawdza kanoniczność oraz pełne granice user space;
5. zwraca zakres półotwarty `[start, end)`.

**\*\*Kryterium akceptacji:\*\*** testy dla `UINT64\_MAX`, adresów niekanonicznych, przejścia przez granicę user/

#### P0.4. Nieatomowy rollback w `aspace\_brk()`

`aspace\_brk()` najpierw tworzy lub rozszerza VMA, a dopiero potem wywołuje `map\_anon\_pages()`. Jeżeli al

**\*\*Plik:\*\*** `kernel/src/mm/process/address\_space.c`, funkcja powiększania sterty.

**\*\*Zadanie:\*\*** zastosować transakcję: najpierw przygotować zakres i zmapować strony, a dopiero po pełnym s

**\*\*Kryterium akceptacji:\*\*** wymuszony brak ramek w połowie operacji pozostawia identyczne VMA, `brk` i lic

### P1 – błądy kontraktów allocatorów

#### P1.1. `kmalloc\_aligned()` nie gwarantuje dowolnego wyrównania

`buddy\_alloc\_aligned()` zaokrągla potrzebny rozmiar do potęgi dwójki i zakłada, że baza buddy jest odpow

**\*\*Pliki:\*\*** `kernel/src/mm/alloc/api/alloc.c`, `kernel/src/mm/alloc/heap/buddy.c`.

**\*\*Zadanie:\*\*** jasno zdefiniować API: albo przyjmować wyłącznie potęgę dwójki i to walidować, albo dodać

#### P1.2. `kfree\_pages()` ignoruje parametr `pages`

`kfree\_pages(void \*ptr, size\_t pages)` ignoruje `pages` i deleguje do `heap\_free()`. To tworzy myślicy kon

**\*\*Plik:\*\*** `kernel/src/mm/alloc/api/alloc.c`.

#### P1.3. `krealloc()` nie przechowuje rzeczywistego rozmiaru danych

Poza debug mode `heap\_usable\_size()` zwraca rozmiar zaokrąglonego bloku buddy, a nie liczbę bajtów danych

#### P1.4. Overflow w alokacji contiguous i DMA

`contig\_bytes\_to\_frames()` i `dma\_bytes\_to\_frames()` wykonują `bytes + ARCH\_PAGE\_SIZE - 1` bez jawnej och

**\*\*Pliki:\*\*** `kernel/src/mm/alloc/special/contiguous.c`, `kernel/src/mm/alloc/special/dma.c`.

**\*\*Kryterium akceptacji:\*\*** największe wartości `size\_t`, `UINT64\_MAX`, długości niebędące wielokrotności

### P1 – paging, TLB i mapowania

Należy przeprowadzić audyt wszystkich funkcji `map`, `unmap` i `protect` pod kątem transakcyjności. Je

**\*\*Zadania:\*\***

Zadanie	Oczekiwany rezultat
Ujednolicić `map_range`/`unmap_range`	Jedna polityka wyrównania, overflow i rollbacku
Dodać statusy bloków	Rozróżnienie `invalid`, `already mapped`, `no memory`, `permission`
Dokonać batch TLB	Jedno `invalpg` na adres lub pełny flush przy przepełnieniu batcha
Dodać shutdown SMP	IPI lub bezpieczny mechanizm synchronizacji aktywnych CR3
Walidować PTE flags	Zakaz USER w mapowaniach kernela i spójne NX/WRITE/DEVICE
Testować aliasy	Mapowanie tej samej ramki pod różne VA z kontrolą uprawnień

### P1 – semantyka `mmap`/VMA

Obecne `mmap()` rozpoznaje głównie `MAP\_FIXED` i kieruje pozostałe przypadki do anonimowego mapowania.

**\*\*Zadania:\*\***

- Walidować dozwolone kombinacje flag i odrzucać nieobsługiwane zamiast je ignorować.
- Rozdzielić VMA na prefix/zakres/suffix przy czyszczeniu `mprotect`.
- Wykonywać wyszukiwanie VMA pod tym samym blokadą co modyfikacja.
- Ustalić, czy mapowanie anonimowe ma być eager czy demand-paged.
- Dodać guard pages dla stosu oraz limit wzrostu stosu.
- Wprowadzić copy-on-write dla `MAP\_PRIVATE`, gdy pojawi się fork/procesy potomne.

## 4. Ulepszenia architektury i wydajności

# .

### 4.1. PMM strefowy zamiast jednej bitmapy dla całego maksimum fizycznego

Obecna bitmapa indeksuje PFN-y aż do najwyższego użytecznego adresu, także przez dziury w fizycznej przestrzeni.

### 4.2. Per-CPU cache dla małych alokacji

Po ustabilizowaniu słaba warto dodać per-CPU magazyn/cache dla najczęstszych klas 16-256 B. Ograniczyć tylko małe.

### 4.3. Lepszy allocator stron dla dużych bloków

Buddy jest dobrym źródłem dużych bloków, ale obecna sterła mapuje i odmapowuje strony przy każdej operacji.

### 4.4. Cache i DMA

`dma\_alloc\_coherent()` potrzebuje jawnego modelu spójności zależnego od architektury. Należy rozróżnić.

### 4.5. Bezpieczne flagi ochrony

Należy ustalić jedną reprezentację ochrony. Obecnie występują osobne zestawy `PROT\_\*` i `virt::\*` flags.

## 5. Diagnostyka i obserwowalność

Debug allocator ma dobre podstawy: magic, poison, canary, leak table i statystyki. Ograniczeniem jest stała.

Ulepszenie	Cel
Rozszerzyć leak table lub użyć hash table	Brak cichego nieledzenia alokacji po przekroczeniu limitu
Dodać `alloc_id`	Jednoznaczne ledzenie życia obiektu
Dodać guard pages dla dużych alokacji	Wykrywanie overrun/underrun poza canary
Raportować ownera cache i CPU	Debugowanie SMP
Dodać histogram rozmiarów	Strojenie klas słab/buddy
Wykrywać niezgodny parę allocator/free	Natychmiastowe raportowanie błędów ownership
Eksportować snapshot MM	Testy regresyjne: free/used, VMA, mapowania, TLB

## 6. Plan implementacji etapami

Etap	Zakres	Priorytet	Definicja ukończenia
A	Wspólne helpery overflow/range/alignment	P0	Wszystkie publiczne API zakresowe używają helperów
B	Spinlocki SMP + kolejno blokady	P0	Test równoległy nie wykazuje race/double allocation
C	Transakcyjny map/unmap/`aspace_brk`	P0	Błąd w połowie operacji nie zmienia stanu
D	Bezpieczny słab z bitmapą obiektów	P0	Double-free i invalid-free są wykrywane
E	Kontrakty aligned/pages/realloc	P1	API ma dokumentowane i testowane gwarancje

```
| F | Overflow DMA/contiguous + strefy PMM | P1 | Testy graniczne przechodz■, DMA32 dzia■a deterministycznie |
| G | Pe■niejszy VMA/mmap/mprotect | P1 | Cz■ciowe zakresy s■ poprawnie dzielone |
| H | TLB shutdown SMP i batchowanie | P1 | Zmiany mapowania s■ widoczne na wszystkich CPU |
| I | Per-CPU cache i optymalizacja buddy/slab | P2 | Benchmark pokazuje mniej s■ kontencji■ bez regresji |
| J | Demand paging/COW/huge pages | P2 | Funkcje maj■ testy integracyjne z procesami |
```

## 7. Minimalny zestaw testów regresyjnych

Należy doda■ testy jednostkowe możliwe do uruchamiania poza j■drem dla bitmapy, regionów, alignmentu i c

```
| Grupa | Przypadki |
|---|---|
| Bitmapa | pusta, pełna, jeden bit, granice 63/64/65, zakres przez granic■ s■owa, double-free |
| PMM | pojedyncze ramki, contiguous, align 0/1/nie-potęga, max address, brak pamięci |
| Buddy | każdy order, koalescencja, fragmentacja, invalid pointer, double-free, overflow align |
| Slab | każda klasa, pełny/pusty slab, double-free, invalid pointer, równoległość |
| API heap | `calloc` overflow, `realloc(NULL)`, `realloc(ptr,0)`, aligned, pages |
| Mapowanie | cz■ciowy b■d, rollback, overlap, unmap fragmentu, protect fragmentu |
| VMA | zakresy graniczne, `brk` rollback, guard page, s■siednie VMA |
| DMA | 32-bit zone, d■ugość overflow, unmap rollback, sync CPU/device |
| Ochrona | NX, SMEP, SMAP, USER/KERNEL PTE, zakaz W^X |
```

## 8. Dokumentacja i CI

Dokumentacja architektury powinna u■ywać ga■zi `stabilizing`, a nie nieaktualnych linków do `unstable`.

Repozytorium powinno mie■ CI, który buduje kernel w tym samym cross-toolchainie co Docker/QEMU, uruchamia

## 9. Kolejność rekomendowana dla autora

Najpierw należy wdrożyć helpery zakresów i overflow, następnie spinlocki SMP. Dopiero potem warto napraw

> \*\*Nie rekomenduj■ jeszcze w■czania `HEAP\_USE\_SLAB` w produkcyjnej ■cie■ce.\*\* Najpierw trzeba doda■ w

## 10. Odwołania do kodu

Najważniejsze ustalenia wynikaj■ bezpośrednio z analizowanej ga■zi:

```
- [API alokacji](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/api/allo
- [Buddy allocator](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/
- [Slab allocator](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/s
- [PMM](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/physical/pmm.c) c
- [Przestrzenie adresowe](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/proce
- [DMA](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/special/dma.c) i
- [Dokumentacja architektury](https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/TrangorgeOS%20
```

## References

```
[1]: https://github.com/CTRL-F-org3/TrangorgeOS/tree/stabilizing "TrangorgeOS – ga■■■ stabilizing"
[2]: https://github.com/CTRL-F-org3/TrangorgeOS/commit/7a0d836f2180874aac3e8c92493c791aec89a8b3 "Trangorg
[3]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/buddy.c "Buddy
[4]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/slab.c "Slab al
[5]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/process/address_space.c "A
[6]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/physical/pmm.c "Phys
[7]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/special/dma.c "DMA a
[8]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/TrangorgeOS%20%E2%80%94%20dokumentacja%20
```

## panic.md

```
Kernel Hang / Panic Report – 2026-10-09
Kernel Hang / Panic Report – LAPIC base sanity check
```

Key findings:

```
- `APIC_BASE_MSR` read reports x2APIC when bit 10 set; else LAPIC assumed at `base_phys` passed in.
- `lapic::init(base_phys)` uses `base_phys` straight as virtual if it's `>= 0xFFFF800000000000`.
- In SMP init, APs are booted with `send_startup_ipi` using `trampoline::TRAMPOLINE_BASE >> 12` as vector
```

Open question:

```
- Where is actual LAPIC physical base loaded from? Current code does not show base-specific usage beyond
```

Next active items:

```
1. Confirm TRAMPOLINE_BASE value and that identity mapping covers it.
2. Confirm `build.rs` linker path behavior for x86_64.
```

### 3. Add serial console capture in QEMU and reproduce hang location.

#### ## Status

Kernel **builds and boots** in qemu (GTK display, 4 vCPUs) but **freezes during runtime**. No console / serial output was captured from the GTK session – the VM stays quiet and appears hung.

Attempted serial capture with qemu (`-serial file:/tmp/qemu_serial_out.txt`) did not produce a readable guest log at the time of this report, so the exact stop point is inferred from the boot path rather than from live kernel prints.

#### ## Boot entry

```
`kernel_main(boot_info: &'static bootloader::BootInfo) -> !` (`src/main.rs`)
```

#### Sequence:

1. ``init()``
2. ``mm::init_from_boot_info(boot_info)`` → “[mm] allocator initialized OK” expected
3. ``init_permissions()``
4. ``gfx::init()`` → “[gfx] framebuffer initialized OK” expected
5. (optional) ``hdmii::init::init()``
6. ``pci::init()``
7. ``nic::runtime::init()``
8. ``bluetooth::init::init()``
9. ``fs::init()``
10. ``cpu::init(boot_info)`` ← SMP bring-up, tight waits per AP
11. ``testing::run_all(TESTS)``
12. ``println!("Welcome in my Galaxy!")``
13. ``gfx::refresh()``, ``terminal::init()``, ``terminal::run()``

#### ## Where the hang manifests

In the GTK qemu session the kernel reaches ``cpu::init(boot_info)`` (step 10) and then stops producing visible progress. The SMP initialization polls for each AP to reach its trampoline entry within a tight per-AP timeout (about 100 ms per AP, 3 APs with 4 vCPUs). If an AP does not become ready in time, the boot path blocks waiting for it without emitting a visible panic in the GTK display.

#### ## Observations from the code

- SMP init lives in ``src/cpu/mod.rs`` / ``src/cpu/smp.rs`` and reaches into ``src/cpu/scheduler/smp/mod.rs`` plus the migration / stopper / balancing subtrees. AP boot uses a trampoline at ``TRAMPOLINE_BASE``.
- The bootloader crate (``bootloader = 0.9`` with ``map_physical_memory`` +

#### ### Changed technical context

- Trampoline base is ``TRAMPOLINE_BASE = 0x8000`` in ``kernel/src/cpu/trampoline.rs`` and ``.set TRAMPOLINE_BASE``
- SMP bringup uses ``send_startup_ipi(apic_id, (trampoline::TRAMPOLINE_BASE >> 12) as u8)``.
- Paging init in ``kernel/src/mm/arch/x86_64/paging.c`` skips identity work when ``boot_phys_offset == ARCH_``
- Kernel MM bridge is compiled by ``kernel/build.rs``, but on x86\_64 it does **not** inject an explicit ``-T``
- Trampoline identity map is done relative to ``TRAMPOLINE_BASE`` before paging is enabled.
- ACPI parsing path uses ``phys_offset`` for RSDP/MADT/FADT reads; MADT can carry ``lapic_base_override`` (ty

#### ### Still blocked

- Cannot reproduce and capture exact panic address or serial log yet in this environment; recommended next

``vga_320x200``) maps physical memory and provides ``BootInfo``. The kernel C bridge (``libmm.a``) is compiled by ``build.rs`` for x86\_64.

- There is **no explicit x86\_64 kernel linker script** in the repo at the
- MADT is already used to read ``lapic_base`` in ``kernel/src/cpu/acpi.rs``; ``lapic_base_override`` is also pa

moment – ``build.rs`` adds ``-T ../riscv64-link.ld`` only for riscv64. On x86\_64 the bootloader’s default layout is used. If that layout does not match kernel assumptions about load address / trampoline placement, early boot can fail silently.

- **566 Rust warnings** remain (Rust 2024 strictness around ``&mut static`` / shared references to mutable statics in ``gfx/console.rs``, ``vga_buffer.rs``, ``fs/mbr.rs``, and several scheduler files). These are not compile errors yet, but some are UB-adjacent and can corrupt early-boot state.
- Current ``cargo run`` uses the GTK display variant, so kernel ``println!`` / panic output is not visible from the host side.

#### ## Likely root causes (ranked)

1. **SMP AP bring-up timeout / deadlock** – an AP never reaches the trampoline

- or does not complete early init within the per-AP timeout, and the BSP waits indefinitely (or until a timeout path that may not panic). This is the most probable hang location with 4 vCPUs / 3 APs.
2. **\*\*Incorrect / incomplete physical memory mapping for the kernel image or trampoline region\*\*** – if ``BootInfo`` mapping is wrong, AP bootstrap code can fault silently without a visible triple fault in GTK mode.
  3. **\*\*Undefined behavior from ``mut static`` / shared-ref patterns\*\*** in early boot (GFX, scheduler, FS) corrupting state that SMP init / tests depend on.
  4. **\*\*Missing explicit kernel linker script for x86\_64\*\*** – the bootloader default layout may not match kernel expectations for section / trampoline placement, causing early code to run at wrong addresses.

## Czerwony punkt (hang point)

- ``cpu::init``(``boot_info``) in ``src/cpu/mod.rs`` / ``src/cpu/smp.rs`` – specifically the per-AP wait loop after the SMP startup IPI(s). If the AP never signals readiness, the BSP waits here and the system hangs.

## Verification needed

- Run qemu with a **\*\*serial console\*\*** (``-serial stdio`` or ``-serial file:/tmp/qemu_serial_out.txt``) and ``< /dev/null`` so host can read kernel prints and any panic message.
- Determine whether the log stops at ``[mm]``, ``[gfx]``, ``pci``, ``nic``, ``bluetooth``, ``fs``, or at ``cpu::init`` / AP bring-up.
- Inspect the SMP trampoline path and per-AP early stack / GDT / IDT setup.
- Validate the bootloader-provided memory map and kernel load / trampoline addresses against kernel expectations.
- Consider adding an explicit x86\_64 linker script if the bootloader layout is not what the kernel assumes.

## Verification status

- Build: clean (exit 0) for ``driverspacelib``, ``driverspace``, ``kernel``.
- Boot: ``cargo run`` builds the bootimage and launches qemu.
- Runtime: hung; no serial console output captured yet in this session.

## Notes

- The GTK display session shows only qemu theme warnings; no kernel console

### Architecture-specific physical-memory offset notes

- On x86\_64, ``mm::init`` receives ``boot_phys_offset`` from ``boot_info.physical_memory_offset`` and passes it
- Paging code in ``kernel/src/mm/arch/x86_64/paging.c`` has an early exit when ``boot_phys_offset == ARCH_DIRECT_MAP_BASE``
- If the bootloader does not map physical memory at the expected offset, the direct map and/or trampoline
- The LAPIC base can come from ACPI MADT; if that value is used directly as a virtual address somewhere w

### Actionable next steps

1. Boot the kernel with ``-serial stdio`` and capture the exact panic / hang point, not just the GTK display
2. If there is no serial access yet, add an early, unconditional serial write right after paging enable a
3. Compare the bootloader-provided ``physical_memory_offset`` with the value the kernel expects before call
4. If APs are not starting, focus on the trampoline region identity mapping and the startup-IPI path, and

## 2026-09-14 – objdump evidence from debug ELF (``kernel``)

- Static data (confirmed in objdump relocation output):
  - ``boot_phys_offset``: ``0x5c6d88``
  - ``boot_phys_offset_valid``: ``0x5c6d80``
- Disassembly shows explicit checks against ``<boot_phys_offset_valid>`` and ``<boot_phys_offset>`` in main, paging, serial, and CPU bringup paths.
- Paging init path includes early-exit / map logic matching the C/Rust preamble already discussed.

### Observability findings

- The debug kernel ELF (``target/x86_64-kernel/debug/kernel``) exists.
- Serial console capture remains the biggest gap:
  - ``/tmp/kernel_serial_out.txt`` is empty.
  - ``/tmp/kernel_serial_err.txt`` was too short to contain a clear panic line.

### Interpretation

- Because the binary clearly looks at ``boot_phys_offset`` / ``boot_phys_offset_valid`` around paging init ``77b2..a5``, the most likely first failures are:
  1. Paging init fault when bootloader-provided offset is incompatible with ``ARCH_DIRECT_MAP_BASE``, OR
  2. SMP bringup fault when AP trampoline/stack/identity mapping is inconsistent with LAPIC/ACPI MADT info.

- Correlation is now feasible if serial output is captured and mapped back with the debug ELF.

#### ## Branch state

- `git status`: modified: `panic.md`; otherwise on `stabilizing`.
- HEAD: `183557c` (`stabilizing`).

output is visible there.

#### ### Current working branch and state

- Branch: `stabilizing` (tracking remote `origin/stabilizing`).
- Working tree has unstaged changes mainly in scheduler / SMP and some target/flycheck artifacts; kernel
  - `kernel/src/cpu/scheduler/collections/mod.rs`
  - `kernel/src/cpu/scheduler/smp/migration/mod.rs`
- The most relevant panic context is in `panic.md`; the key code references are `kernel/src/main.rs`, `ke

#### ### Current working branch and state

- Branch: `stabilizing` (tracking remote `origin/stabilizing`).
- Working tree has unstaged changes mainly in scheduler / SMP and some target/flycheck artifacts; kernel
  - `kernel/src/cpu/scheduler/collections/mod.rs`
  - `kernel/src/cpu/scheduler/smp/migration/mod.rs`
- The most relevant panic context is in `panic.md`; the key code references are `kernel/src/main.rs`, `ke

#### ## TODO

##### ## References

##### ## References

- [ ] Capture reliable serial console output with `-serial stdio`/`-serial file:`.
- [ ] Add an early unconditional serial write after paging enable, before MM/SMP init.
- [ ] Compare bootloader `physical\_memory\_offset` vs `ARCH\_DIRECT\_MAP\_BASE` and LAPIC direct-map expectations.
- [ ] Confirm trampoline identity mapping covers AP trampoline code/stack.
- [ ] If serial log appears, correlate RIP with `kernel` ELF using addr2line/objdump.
- [ ] If offset mismatch is confirmed, decide whether to support dynamic offset or force early remap before paging enable.
- [ ] Fix `kernel/src/cpu/scheduler/collections/mod.rs` and `kernel/src/cpu/scheduler/smp/migration/mod.rs` (`pub pub mod ...`) so `cargo build --release` can produce the release bootimage again.

- `kernel/src/main.rs`
- `kernel/src/cpu/smp.rs`
- `kernel/src/cpu/lapic.rs`
- `kernel/src/cpu/acpi.rs`
- `kernel/src/mm/mod.rs`
- `kernel/src/mm/arch/x86\_64/paging.c`
- `kernel/src/mm/arch/x86\_64/memory.c`
- `kernel/src/cpu/trampoline.s`
- `kernel/src/cpu/trampoline.rs`
- `panic.md`

This report is intended as a continuation note for kernel hang debugging on x86\_64 and is not yet a root-

- `kernel/src/main.rs`
- `kernel/src/cpu/smp.rs`
- `kernel/src/cpu/lapic.rs`
- `kernel/src/cpu/acpi.rs`
- `kernel/src/mm/mod.rs`
- `kernel/src/mm/arch/x86\_64/paging.c`
- `kernel/src/mm/arch/x86\_64/memory.c`
- `kernel/src/cpu/trampoline.s`
- `kernel/src/cpu/trampoline.rs`
- `panic.md`

This report is intended as a continuation note for kernel hang debugging on x86\_64 and is not yet a root-

- [ ] Add a serial-handshake early print right after paging enable, before SMP bringup.

- [ ] Add a small test mode to boot with QEMU and a serial file/stdio in CI or locally, so the hang can be reproduced
- [ ] Compare actual bootloader-provided `physical_memory_offset` with the offset used by paging init and ensure they match
- [ ] Confirm trampoline identity map covers at least the trampoline code and stack range used by AP bringup